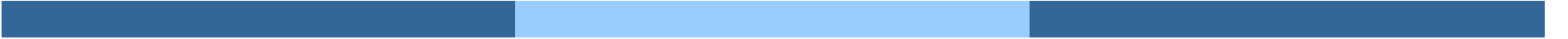


File System



File System e Memoria di Massa

- I File System forniscono metodi efficienti di accesso ai file
- Organizzano l'informazione in termini di **File**
- Il file system risiede permanentemente in memoria di massa
- Tipicamente questa memoria è fornita dai dischi:
 - Possono essere letti e scritti localmente (scarico un blocco in memoria, scrivo il blocco, ricarico il blocco)
 - Permettono l'accesso, sia sequenziale che diretto, ai blocchi fisici di memoria che memorizzano i diversi file.
- La dimensione dei blocchi varia da dispositivo in dispositivo
 - Da 512 a 4096 Byte
 - Solitamente 4096 per NVM

Concetto di File

- Unità logica di immagazzinamento in memoria secondaria
- Vista uniforme sull'informazione memorizzata in mem non volatile
- Spazio contiguo di indirizzi logici
- Diversi tipi di dato:
 - Dati
 - ▶ numerico
 - ▶ caratteri
 - ▶ binari
 - ▶ programmi (sorgente, eseguibile, etc.)
- Contenuti definiti dal creatore del file
 - Diversi tipi
 - ▶ Considera **text file, source file, executable file**

Attributi del File

- Un file è associato ad un insieme di attributi
 - **Name** – denominazione (per utente umano)
 - **Identifier** – tag unico (numero) che identifica il file nel file system
 - **Type** – necessario per sistemi che supportano tipi diversi
 - **Location** – pointer alla locazione del file sul device
 - **Size** – dimensione corrente
 - **Protection** – chi può fare reading, writing, executing
 - **Time, date, and user identification** – dati per protection, security e usage monitoring (creazione, modifica, uso)

- Diverse variazioni ed estensioni

- Su disco viene mantenuto un indice del file (**inode** in Unix/Linux) con i metadati del file
 - non il nome del file delegato alle directory entries

Operazioni File

- File è un **tipo di dato astratto**
- Occorre definire il tipo di operazioni (associate a system call)
 - **Create** – creazione del file
 - **Open(F_i)** – cerca il file su disco con nome F_i , controlla i permessi, restituisce un riferimento per le altre operazioni
 - **Close (F_i)** – chiude il file
 - **Write** – per la scrittura mantiene un **write pointer**
 - **Read** – mantiene un **read pointer**
 - **Reposition within file - seek**
 - **Delete** – cancellazione del file
 - **Truncate** – cancellazione del contenuto del file mantenendo gli attribute (size diventa zero)

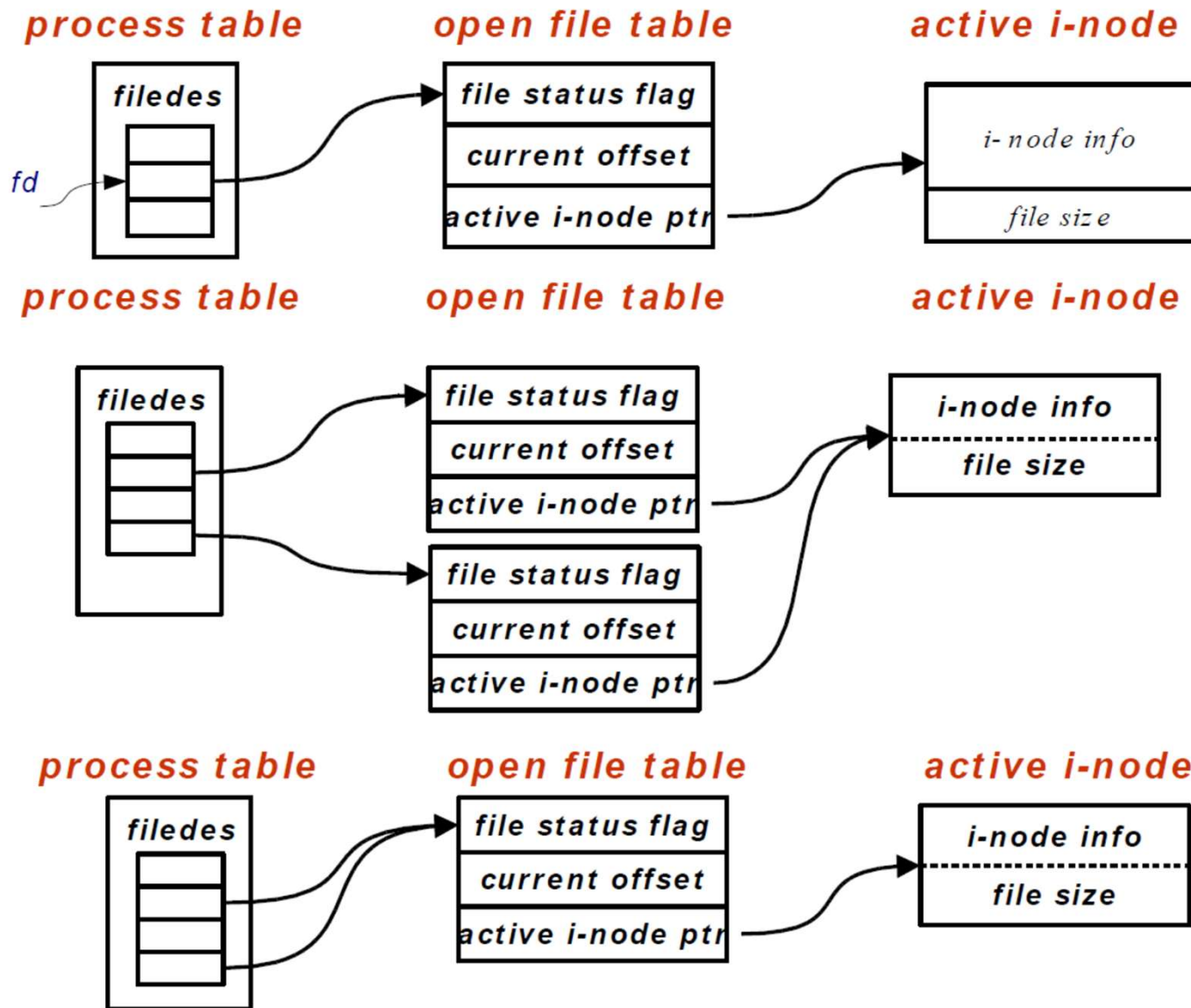
Open File

- Per evitare di scorrere ogni volta il File System si apre il file prima di usare altre operazioni (non create e delete)
- Con open si mantengono in memoria principale le info sul file aperto
 - **Open-File Table:** SO traccia tutti file aperti, i processi hanno info sui loro file aperti (due tabelle, una per-processo e una di sistema)
 - ▶ Entry nella **tabella per-processo**
 - **File descriptor:** numero intero indice nella tabella dei file aperti per processo assegnato quando il file viene aperto
 - Punta alla entry dei file aperti nella tabella di sistema
 - ▶ Entry nella **tabella di sistema:**
 - Descrive come viene aperto il file e dove sta leggendo/scrivendo un processo
 - Punta al File Control Block
 - **File Control Block (FCB):**
 - ▶ Contiene i metadati dei file
 - **Attributi del file**
 - **Locazione del file su disco:** informazione per l'accesso diretto al file, senza dover cercare attraverso le directory

Strutture del File System

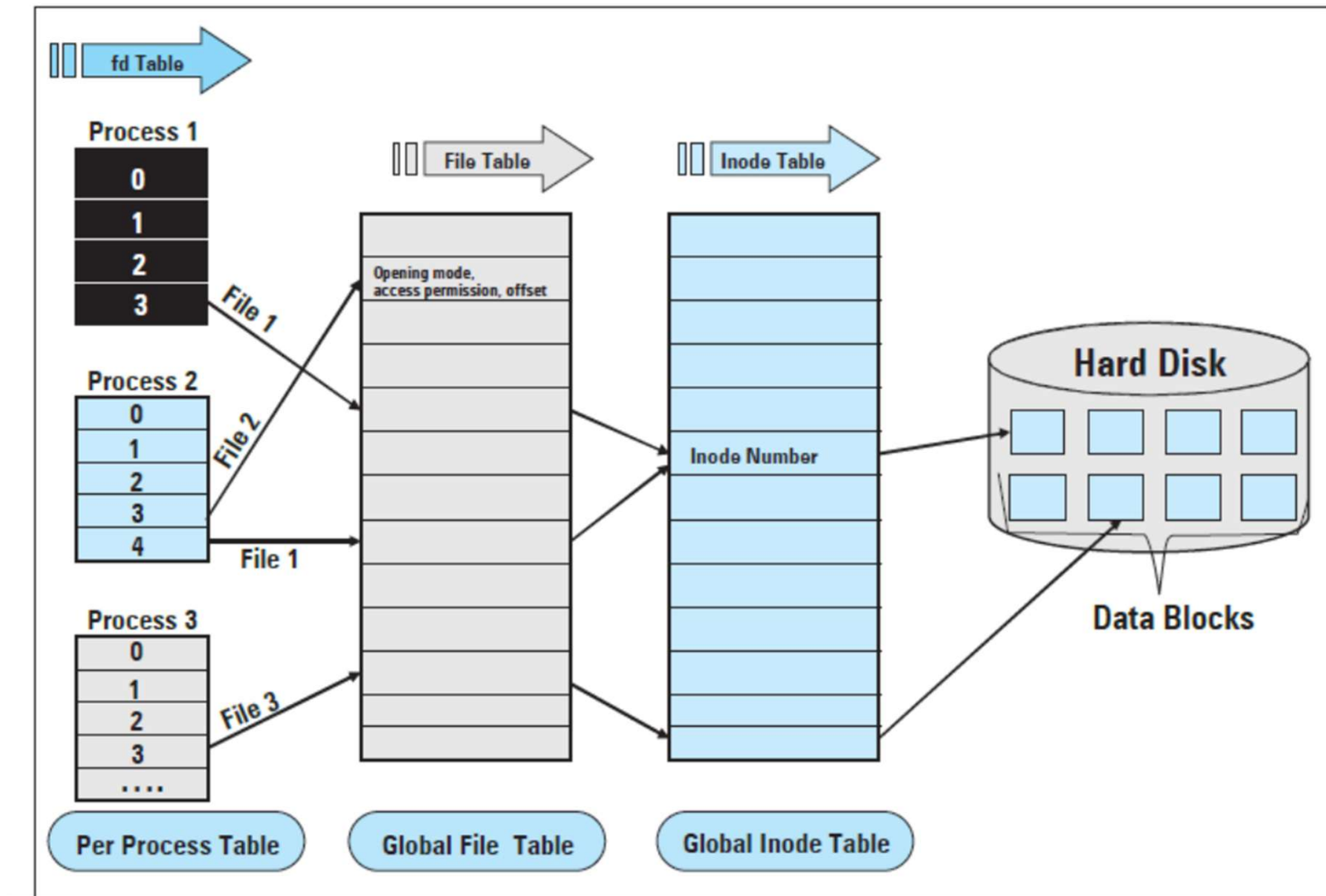
- Strutture del file system in memoria (Linux)

FCB



Strutture del File System

- Strutture del file system in memoria (linux)



Tipo di File, Nomi, Estensioni

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Struttura dei File

- Se si assumono diverse strutture di file, ogni tipo richiede algoritmi diversi
- Struttura minimale (UNIX/Linux)
- UNIX/Linux – sequenza lineare di bytes (programmi gestiscono da soli)
 - Ogni byte del file viene indirizzato con offset,
 - Ogni blocco logico è 1 byte che è contenuto su un blocco del disco
- I file si definiscono in blocchi logici, ma si mappano su blocchi del disco
 - Se file di 1949 byte con blocchi di 512 byte su disco, si allocano 4 blocchi, cioè 2048 byte con spreco di 99 (frammentazione interna)
- Il file system Linux/Unix poi distingue diversi tipi di file: ordinari, directory, speciali, link, etc. (tutti di base sono sequenze lineari di byte)

Metodi di Accesso

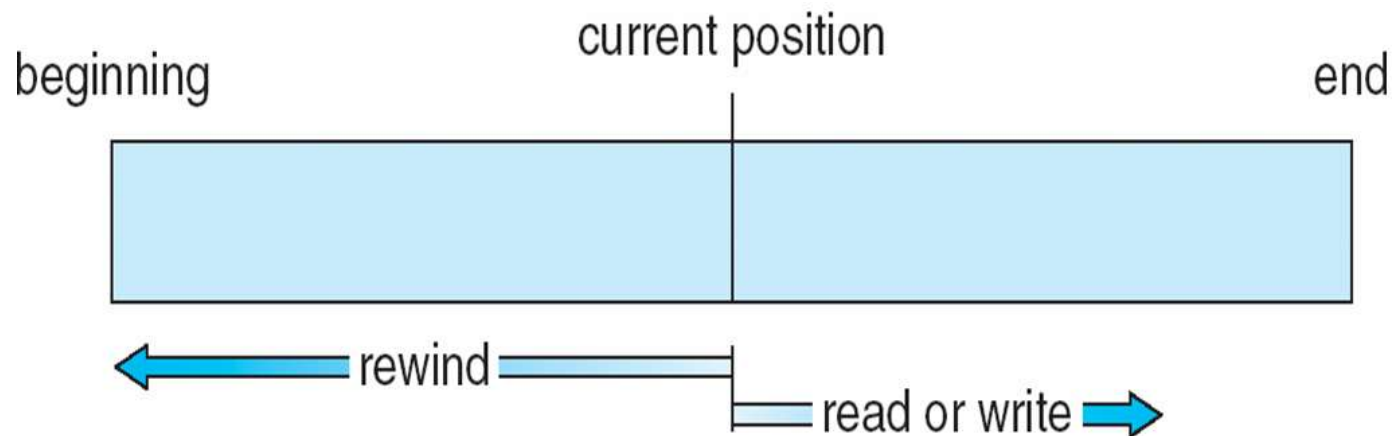
- Diverse possono essere le modalità di accesso di un file
- **Accesso sequenziale** (modello del nastro)

read next

write next

reset

Funziona sia su dispositivi ad accesso sequenziale, sia random



Metodi di Accesso

- Diverse possono essere le modalità di accesso di un file
- **Accesso sequenziale**

```
read next  
write next  
reset
```

- **Accesso diretto (relativo) – modello disco**
file è assunto di lunghezza fissata composto di **logical records**

```
read n  
write n  
position to n  
        read next  
        write next  
rewrite n
```

n = **numero di blocco relativo**

Simulazione di accesso sequenziale su Direct-access File

- Diverse possono essere le modalità di accesso di un file
- **Accesso sequenziale simulato con l'accesso diretto**

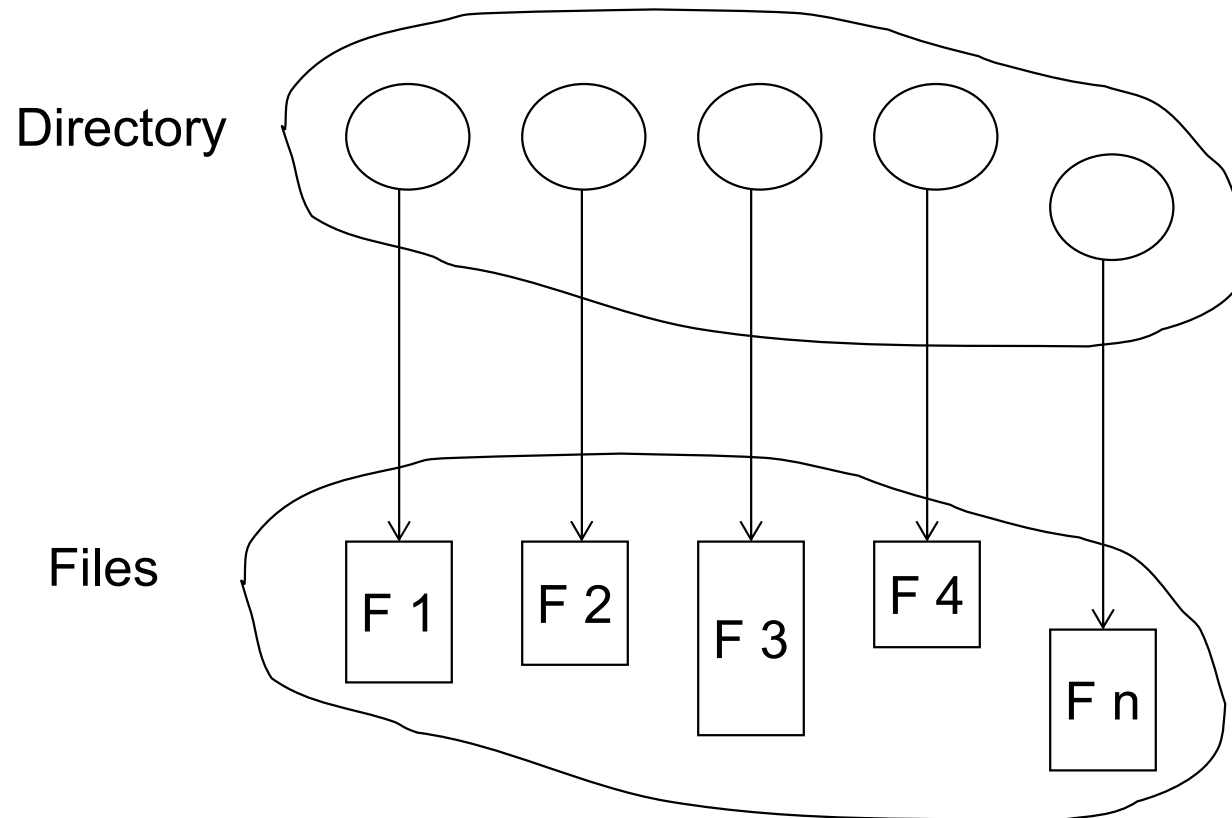
sequential access	implementation for direct access
<i>reset</i>	<i>cp = 0;</i>
<i>read next</i>	<i>read cp;</i> <i>cp = cp + 1;</i>
<i>write next</i>	<i>write cp;</i> <i>cp = cp + 1;</i>

Altri Metodi di Accesso

- Con accesso diretto si possono definire altri metodi di accesso
- Creazione di **indici** per i file
 - Mantiene indici in memoria per un veloce determinazione della locazione dei blocchi da processare (puntatori ai blocchi)
 - Prima cerca indice, poi blocco, poi record
 - Ricerca veloce per file grandi senza troppi accessi
 - Se troppo grande, indice (in memoria) dell'indice (su disco)

Struttura della Directory

- ❑ Le directory possono essere viste come tavole dei simboli per tradurre i nomi dei file nei loro file control block
- ❑ Una collezione di nodi contenente informazione su tutti i file



La directory structure e il files residente su disco

Operazioni su Directory

- Le directory possono essere viste come tavole dei simboli per tradurre i nomi dei file nei loro file control block

- Le directory devono permettere diverse operazioni:
 - Ricerca di un file
 - Creazione di un file
 - Cancellazione di file
 - Lista del contenuto di una directory
 - Rename di un file
 - Spostamenti sul file system

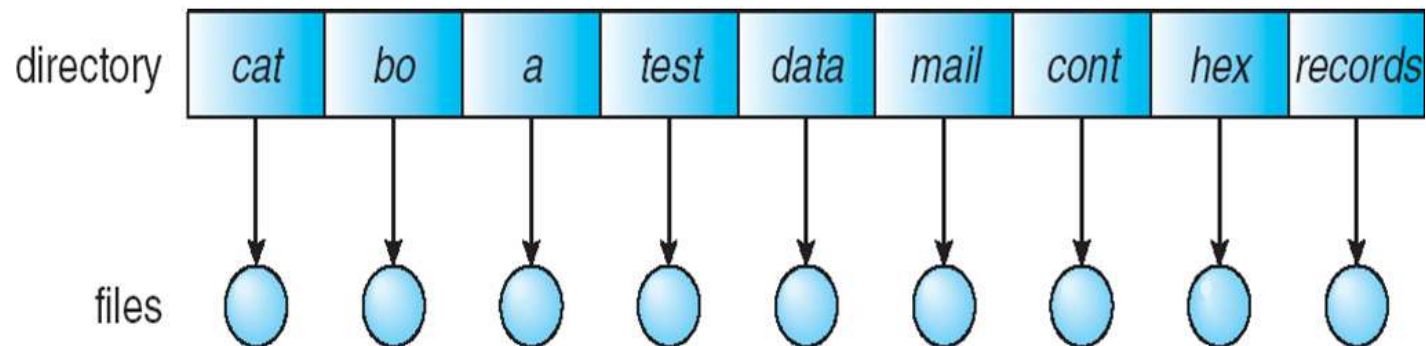
Organizzazione Directory

La struttura delle directory può essere organizzata logicamente per ottenere

- **Efficienza** – locazione veloce dei file
- **Naming** – utile per gli utenti
 - Due utenti possono avere stesso nome per file differenti
 - Lo stesso file con nomi differenti
- **Grouping** – raggruppamento logico di files per proprietà, (e.g., tutti i programmi Java, tutti i game, etc. ...)

Single-Level Directory

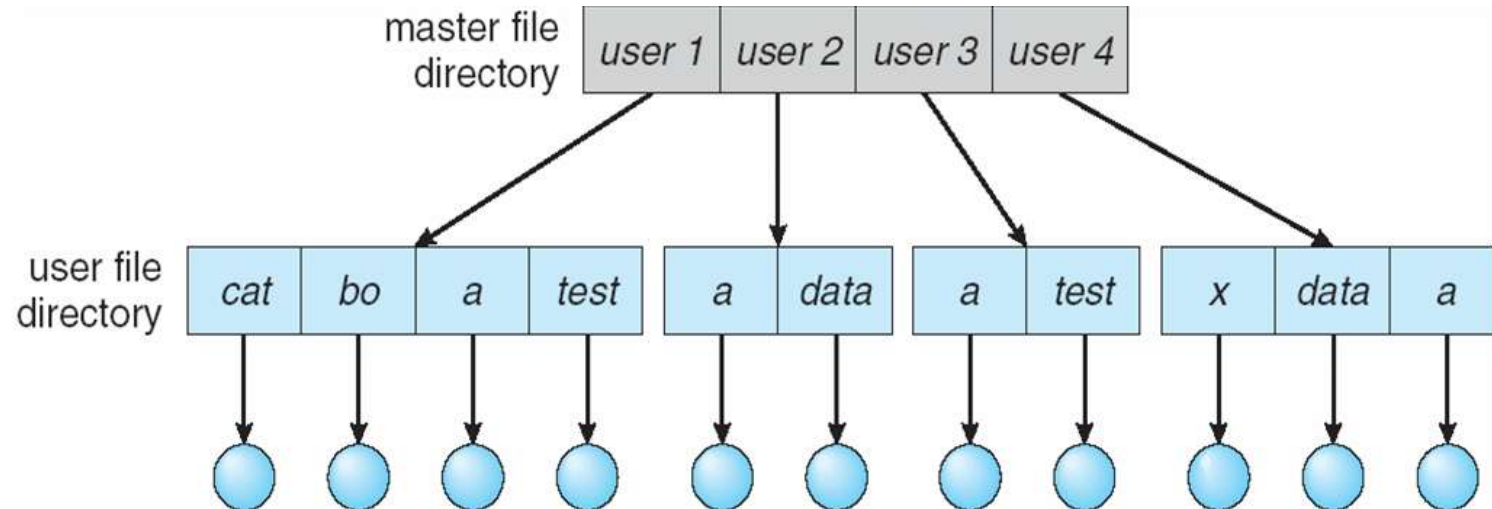
- Una singola directory per tutti gli utenti



- Problema del naming
- Problema del Grouping

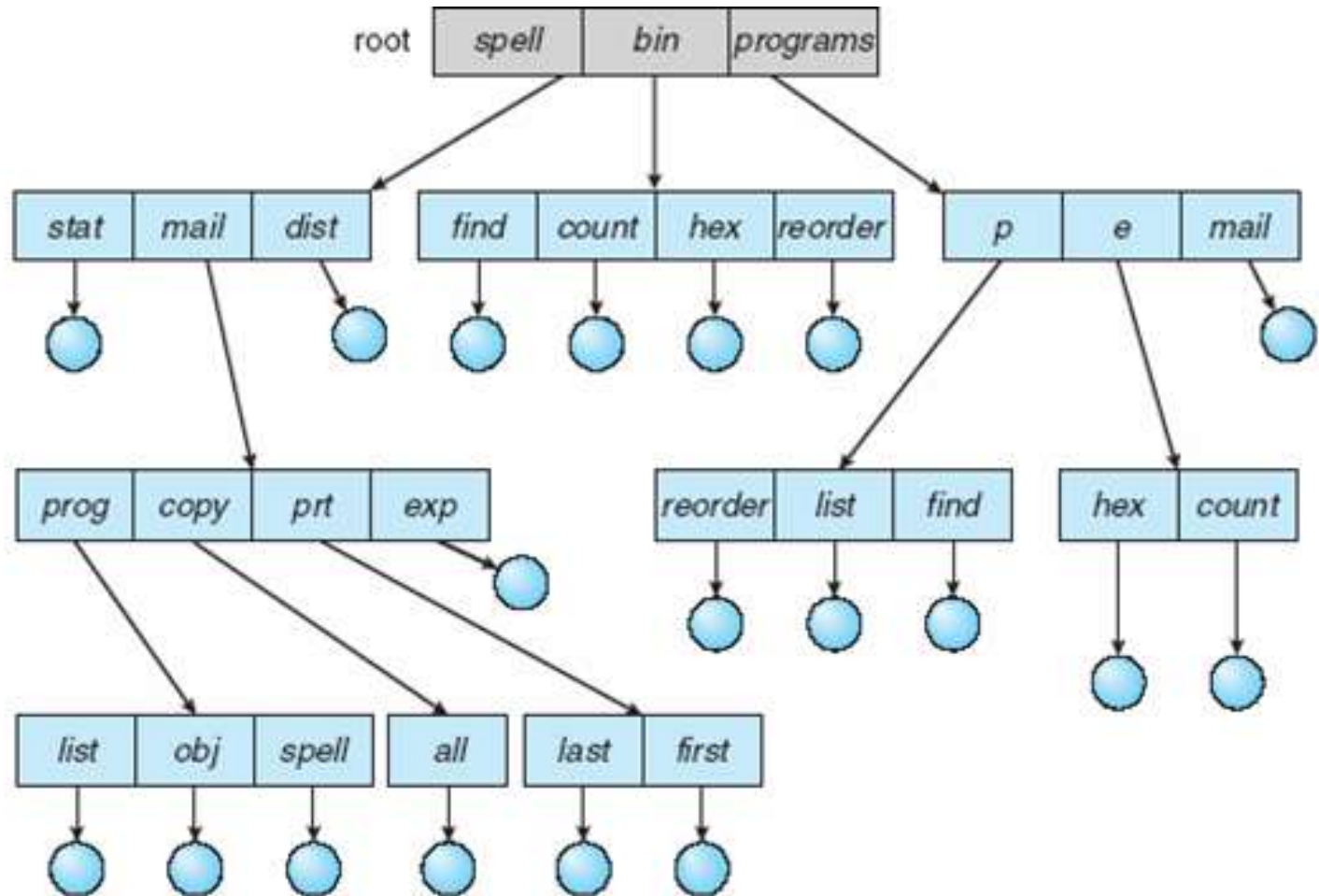
Two-Level Directory

- Directory separate per ogni user



- Path name
- Si può avere lo stesso nome di file name per utenti differenti
- Ricerca efficiente
- Non si può fare grouping

Tree-Structured Directory



Tree-Structured Directory

- ❑ Ricerca efficiente
- ❑ Capacità di grouping
- ❑ Directory attuale (working directory)
 - ❑ `cd /spell/mail/prog`
 - ❑ `type list`

Tree-Structured Directories

- path name **assoluti** o **relativi**

- Creazione di un nuovo file fatta nella directory corrente

- Cancellazione di file

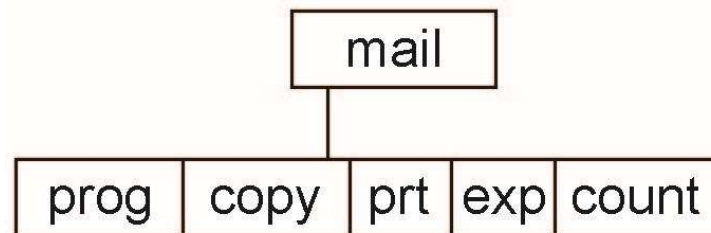
rm <file-name>

- Creazione di una nuova subdirectory è fatto nella directory corrente

mkdir <dir-name>

Esempio: se la directory corrente è **/mail**

mkdir count

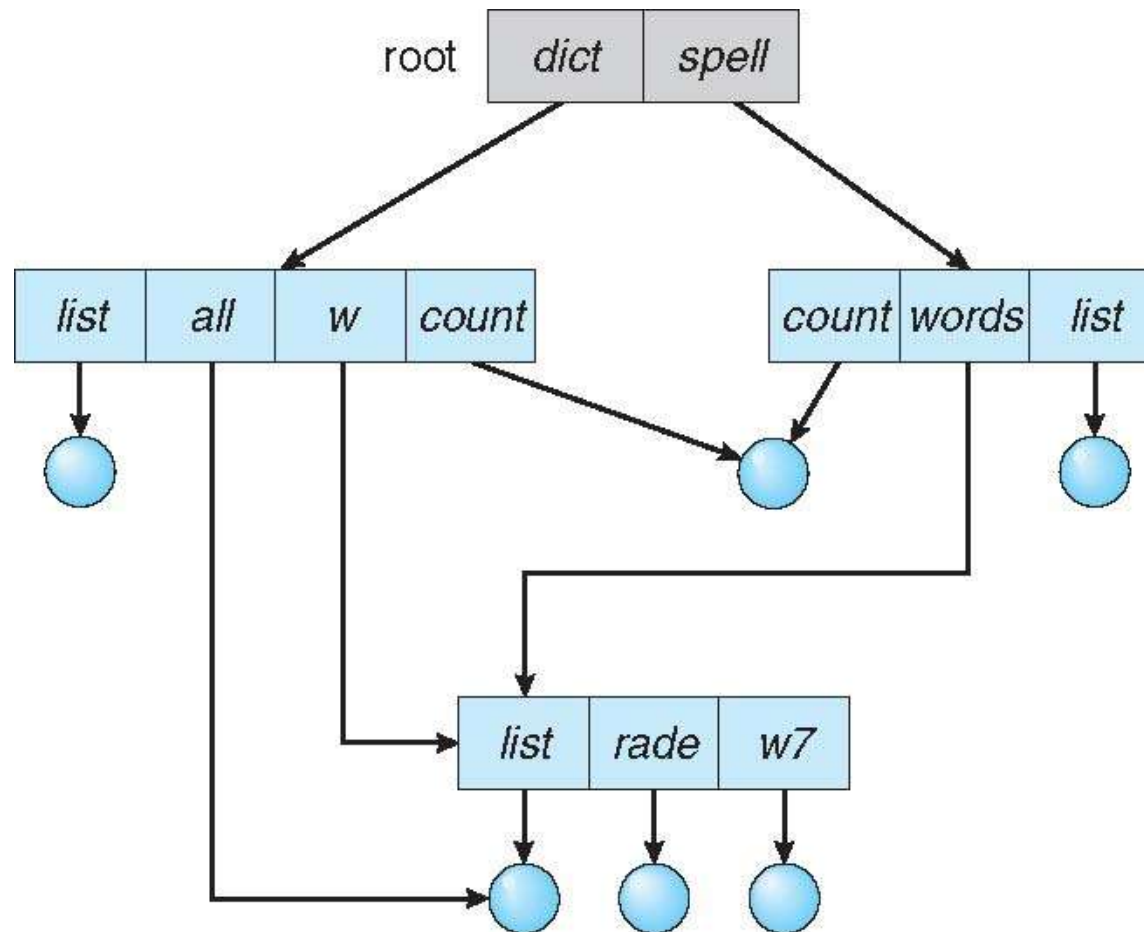


Cancellando “mail” $\Rightarrow$ si cancella l'intero subtree con radice in “mail”

Accesso di utenti nelle directory di altri consentita

Acyclic-Graph Directory

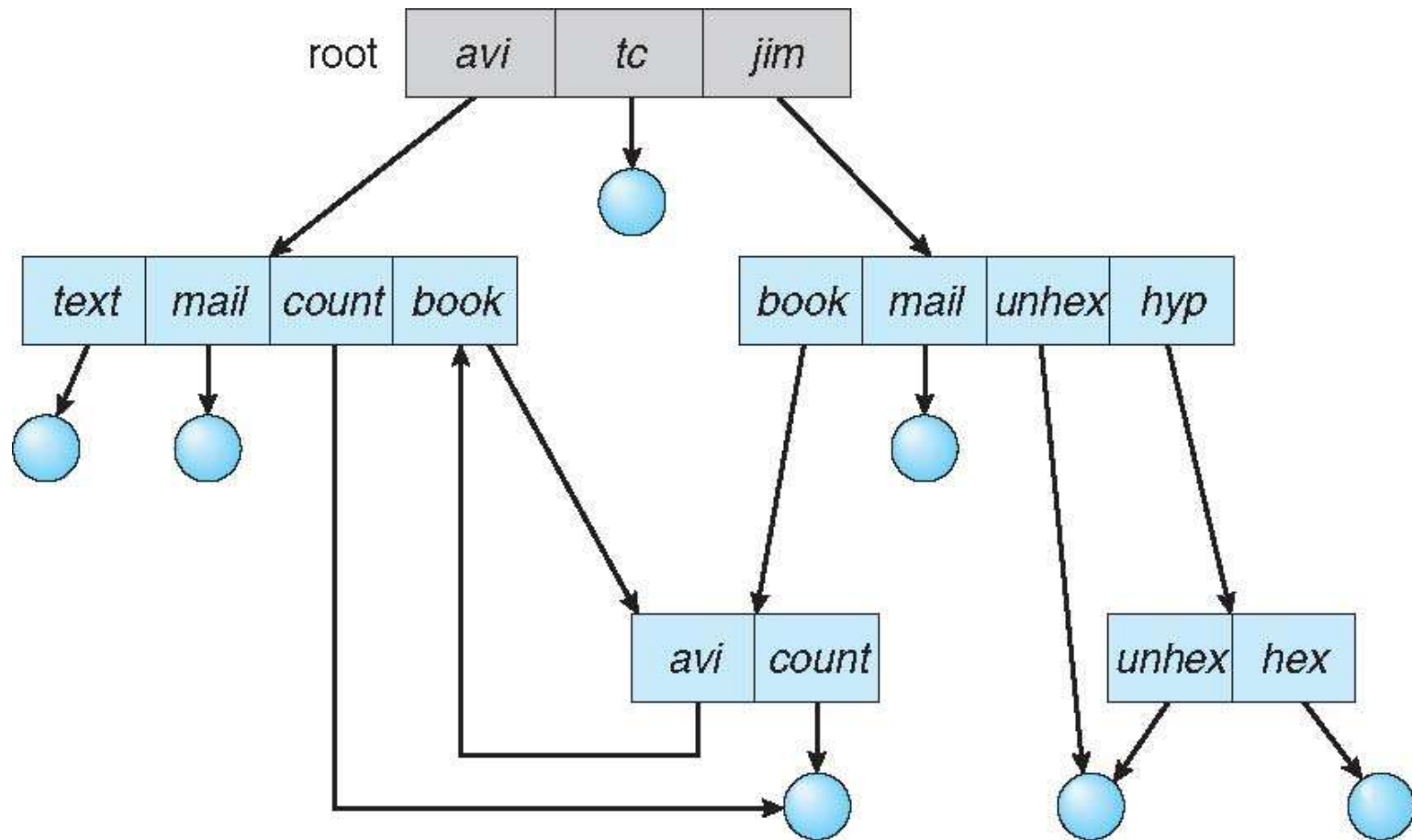
- Ha sub-directory e file condivisi



Acyclic-Graph Directory

- Due nomi differenti (aliasing)
- Nuovo tipo di entry per directory
 - **Link** – un altro nome (pointer) ad un file esistente
 - **Risolvere il link** – segue il pointer per localizzare il file
- Se si cancellano i file subito $\Rightarrow$ dangling pointer per i link
 - Se si mantengono accesso a file non esistenti possibili problemi
 - ▶ Per link simbolici si lasciano i link rotti e SO si accorge del problema
 - Si possono cercare tutti i link associati ad un file ed eliminarli
 - ▶ Occorre memorizzare per ogni file i riferimenti al file, costoso
 - Soluzione con contatore
 - ▶ Si contano i link per file, quando il contatore va a zero si può cancellare il file

General Graph Directory



General Graph Directory

- Se ci sono cicli la situazione è più complessa
 - Ogni ciclo è un nuovo riferimento allo stesso file
 - Ricerca di un file che non termina
 - Per cancellazione contatore può non essere zero anche se tutti i link cancellati per conteggio di auto-riferimenti
- Come garantiamo che non si hanno cicli?
 - Garbage collection
 - ▶ Scansione del file system per verificare l'accessibilità, tutto quello che non è accessibile diventa free
 - ▶ Costoso
 - Mantenere struttura aciclica
 - ▶ Ogni volta che un nuovo link è aggiunto usa un algoritmo di cycle detection per determinare se è OK
 - ▶ Costoso, si evita il ciclo
 - Ignorare/bypassare i link a directory durante le operazioni
 - ▶ Problematici soprattutto i link simbolici

File System UNIX: Caratteristiche

- ❑ Struttura gerarchica
- ❑ Files senza struttura (byte streams)
- ❑ Protezione da accessi non autorizzati
- ❑ Semplicità di struttura

"On a UNIX system, everything is a file; if something is not a file, it is a process."

File Unix

I tipi principali di File sono:

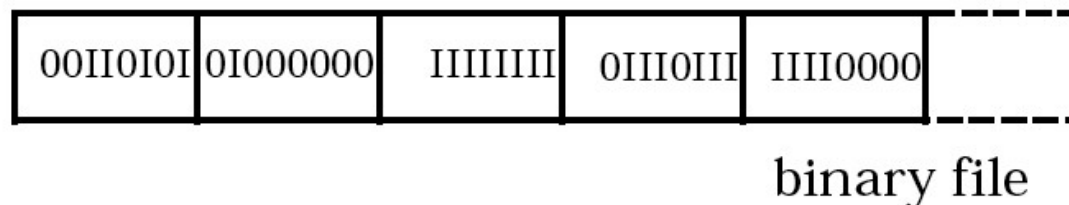
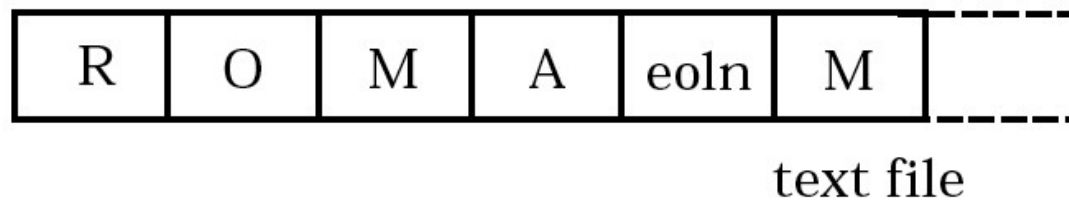
- **File ordinari**
- **Directory**
- **File Speciali**

Il sistema assegna biunivocamente a ciascun file un identificatore numerico, detto

i-number ("index-number"), che gli permette di rintracciarlo nel file system.

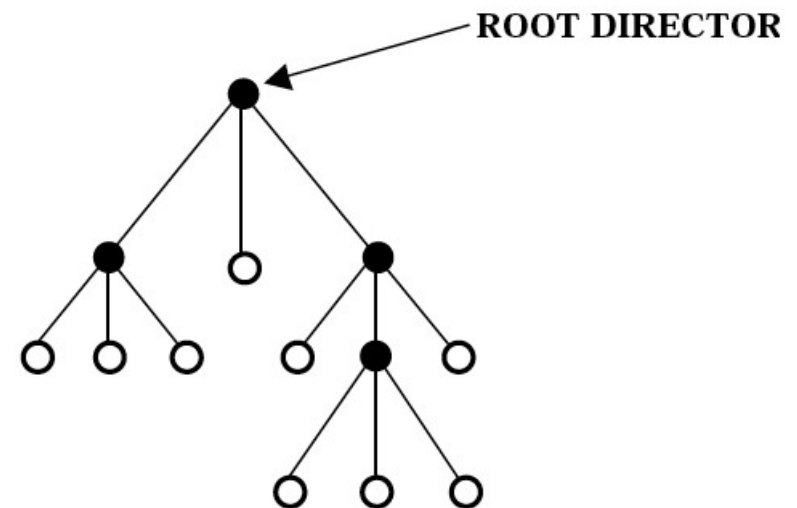
File Ordinari

- Sono sequenze di byte (byte streams)
- Possono contenere informazioni qualsiasi (dati, programmi sorgente, programmi oggetto,...)
- Il sistema non impone alcuna struttura



Organizzazione dei File in UNIX

Per consentire all'utente di rintracciare facilmente i propri files, Unix permette di raggrupparli in cartelle, dette **Directories**, organizzate in una (unica) struttura gerarchica:

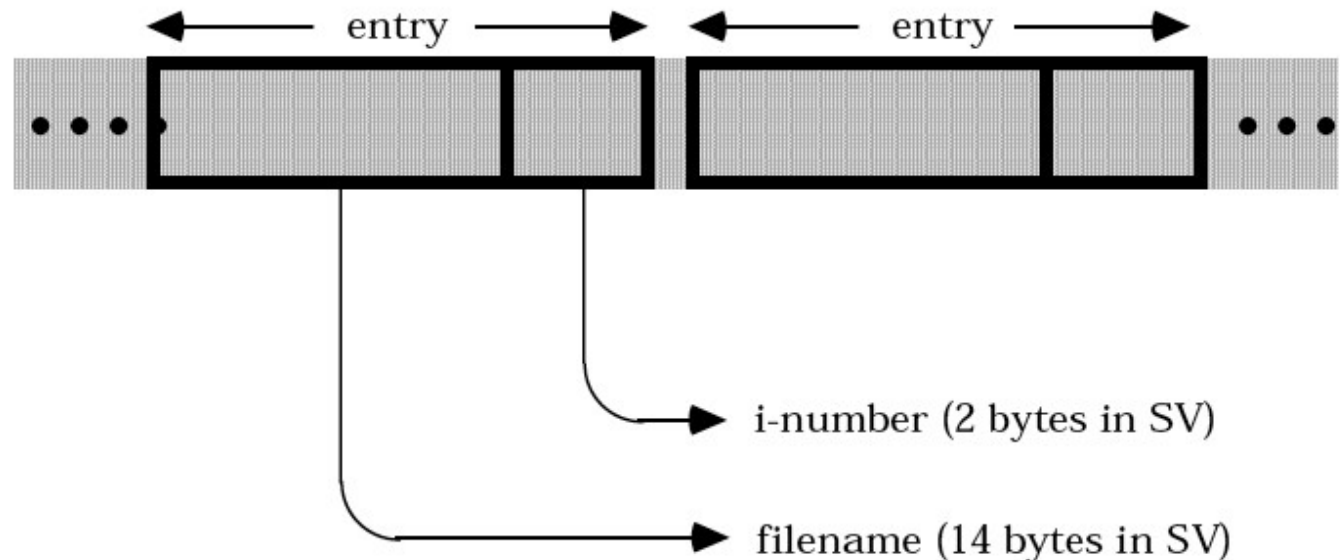


● : directory

○ : file ordinario
directory (vuota)
file speciale

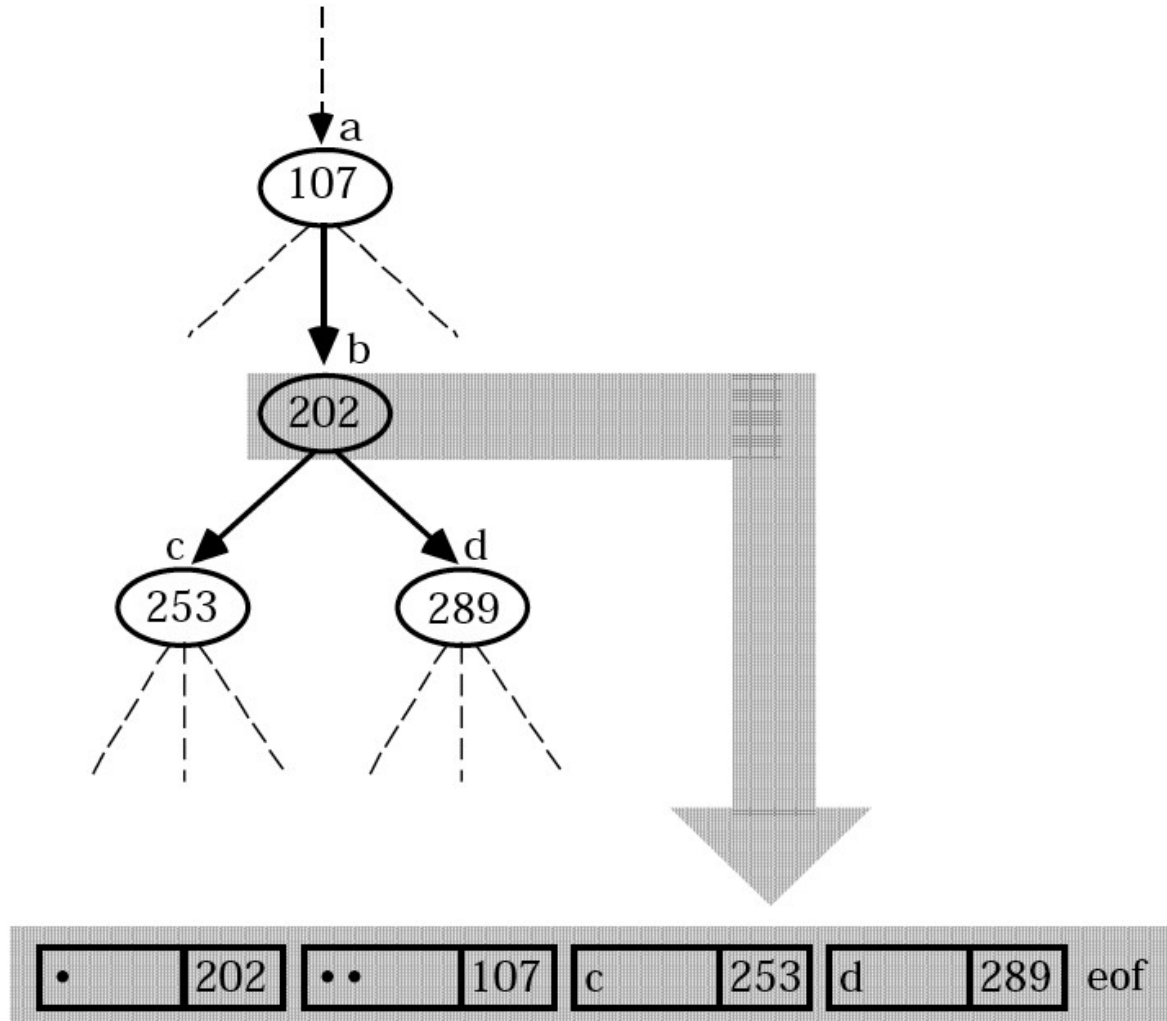
Directories in Unix

- Sono sequenze di bytes come i file ordinari;
- Differiscono dai file ordinari solo perché non possono essere scritte da programmi ordinari
- Il loro contenuto è una serie di **directory entries**:
associazione fra gli i-number (usati dal sistema) e i **filename** mnemonici (usati dall'utente):



In System V (SV), in Linux ext3 solitamente i-num 4 byte e filename fino a 255 byte

Esempio

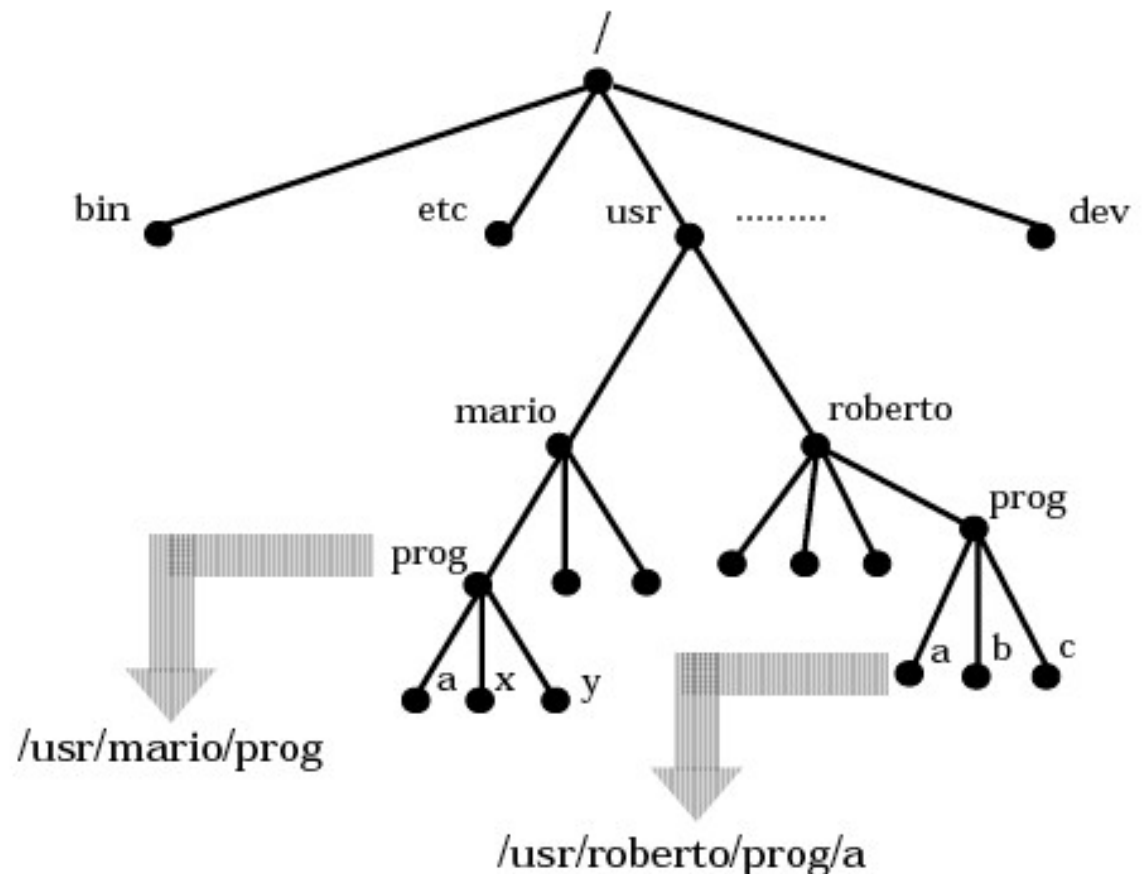
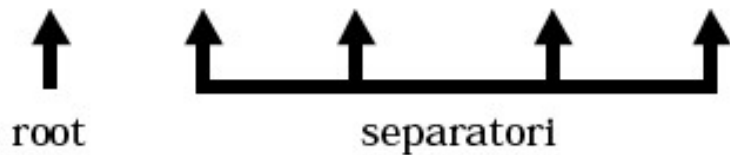


Almeno due entry: la directory stessa “.”, la directory padre “..”

Pathnames

Ogni file viene identificato univocamente specificando il suo **pathname**, che individua il cammino dalla root-directory al file:

`/dir/dir/.../dir/filename`

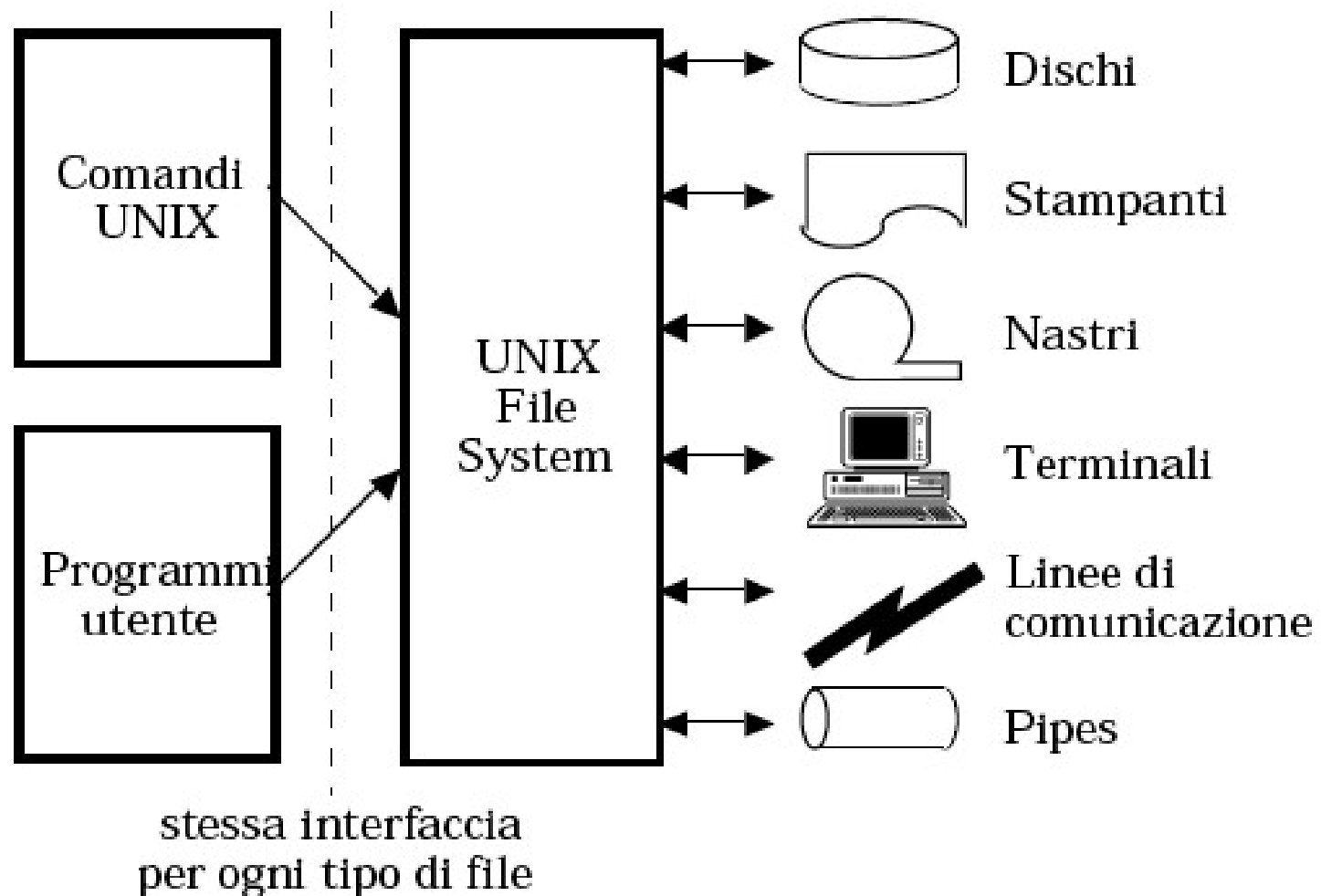


Tipi di File

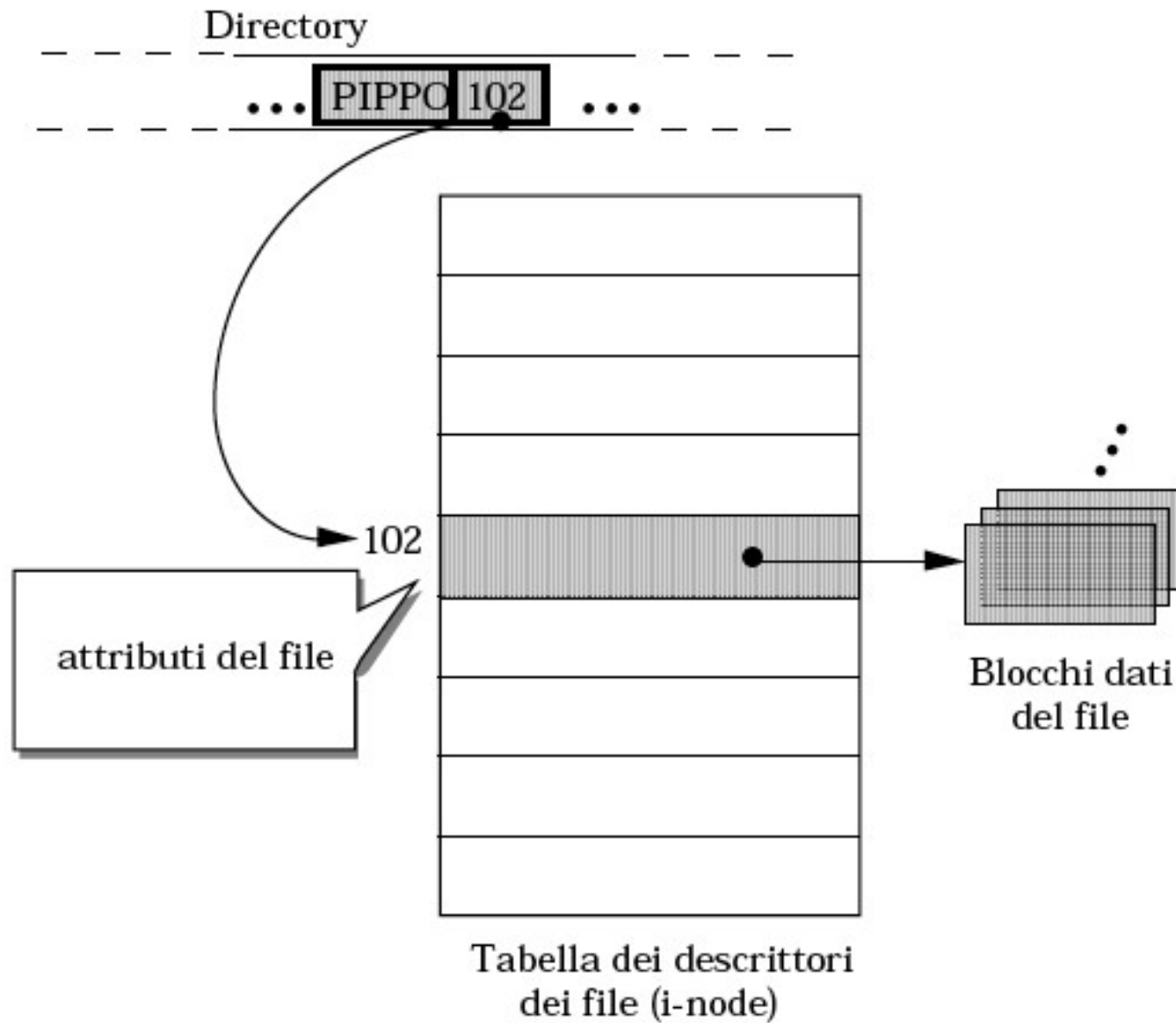
- File regolari
- Directory file
- Link file
- Character special file
- Block special file
- Socket file
- Named pipe file

Files Speciali: vantaggi

- Trattamento uniforme di files e devices
- In Unix i programmi non sanno se operano su un file o su un device

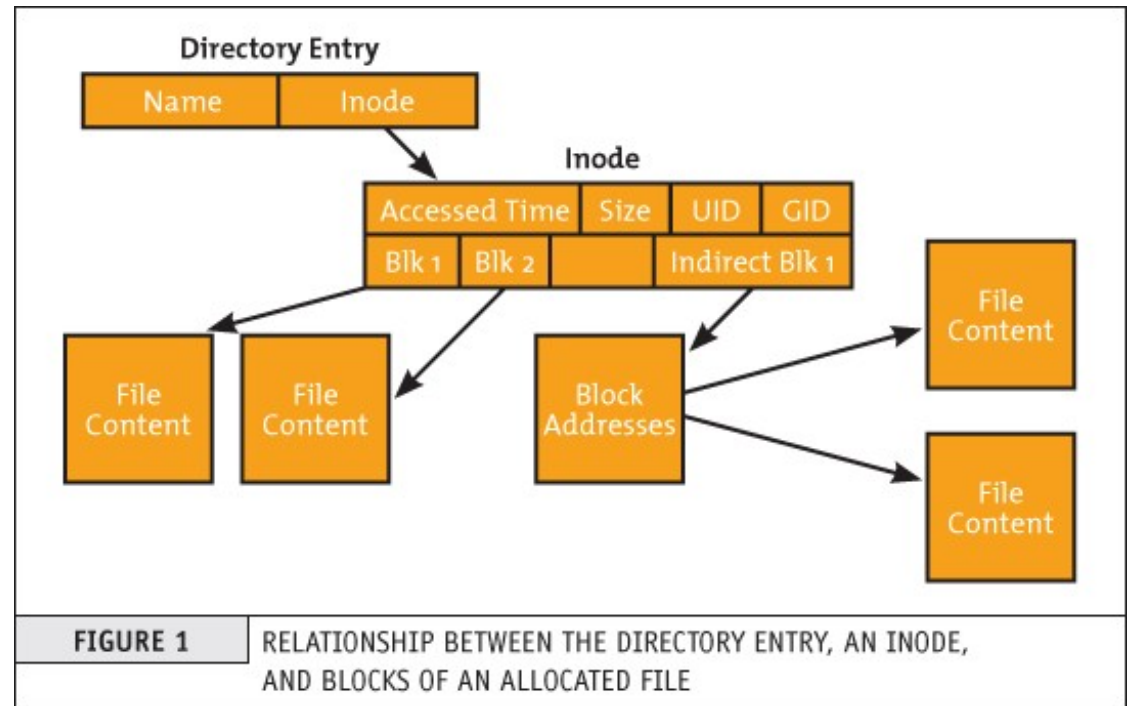
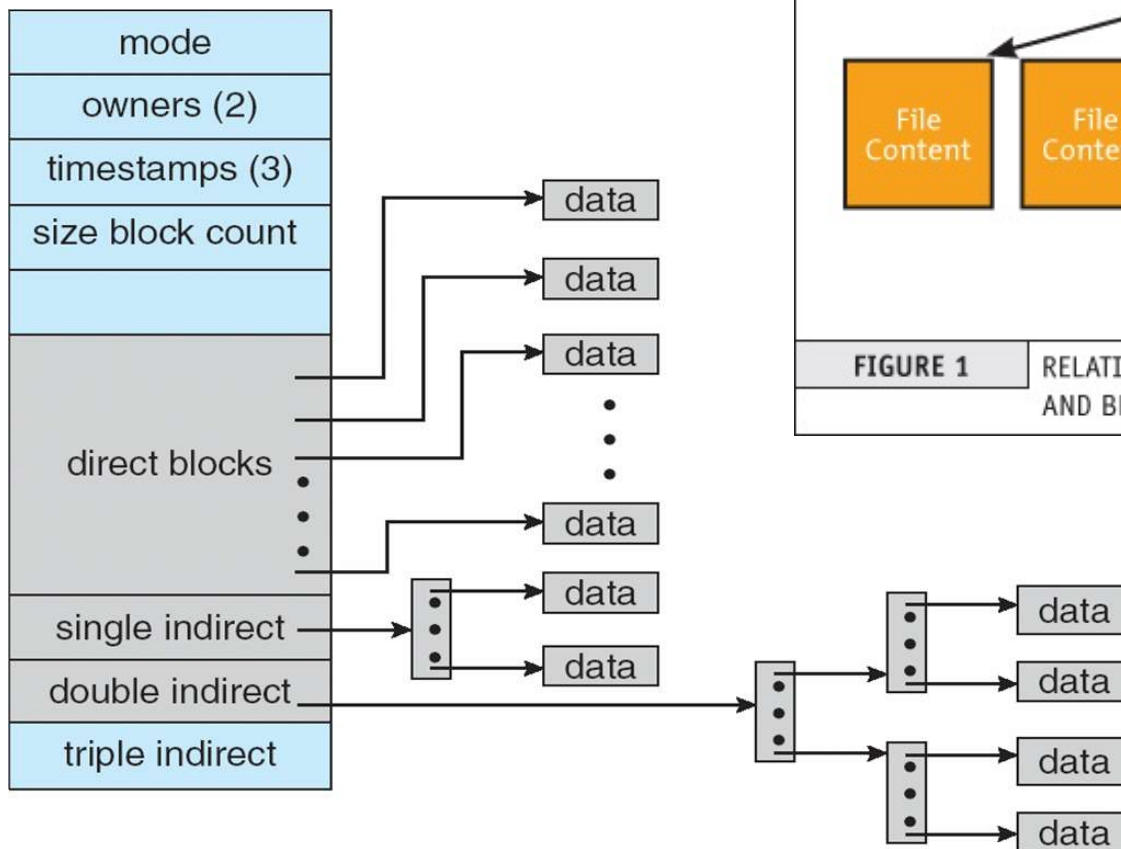


Implementazione File



Linux File System

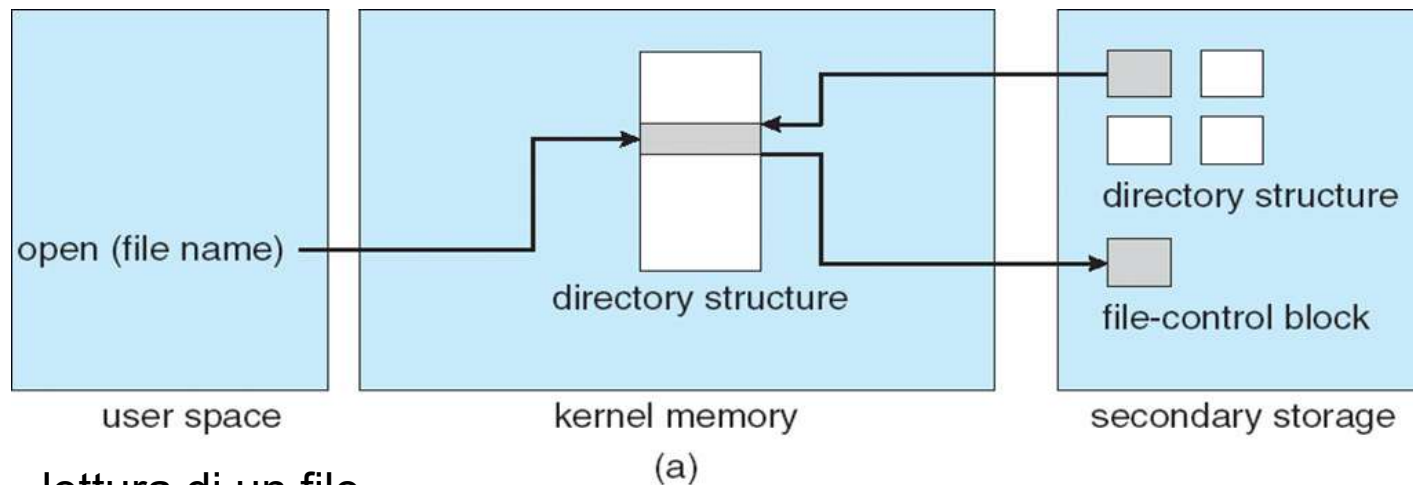
- Inode
- Directory entry
- Block addresses



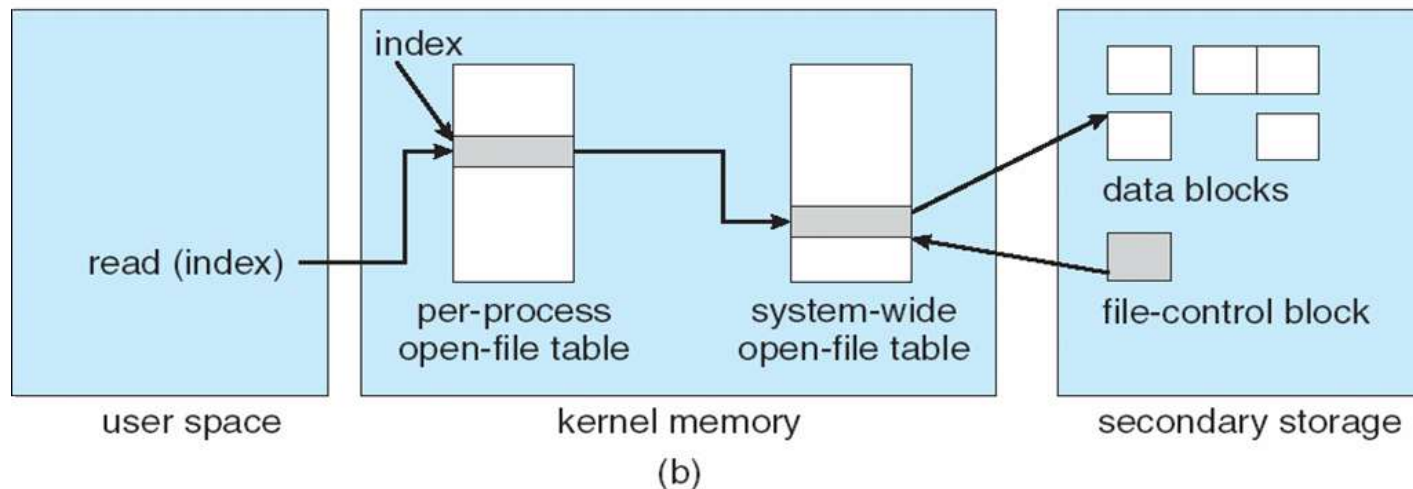
Strutture del File System

□ Strutture necessarie del file system fornite dal SO

□ apertura di un file



□ lettura di un file

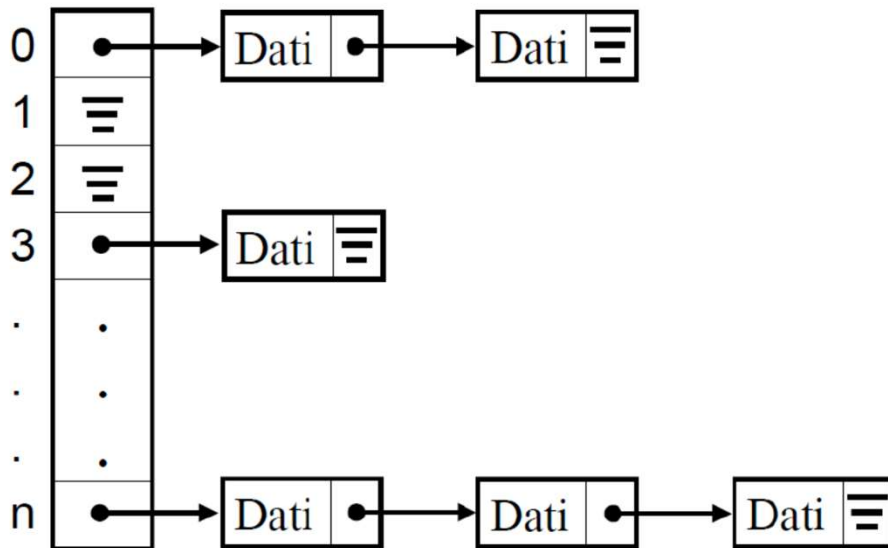


Implementazione Directory

- Implementazione delle directory impatta molto sulle prestazioni
 - Algoritmi di allocazione e gestione delle directory
- Modo più semplice di implementare le directory è con una **lista lineare** un array contenente identificatore del file e puntatore al FCB
 - Facile da programmare (si cerca il file e si accede)
 - Poco efficiente da eseguire
 - ▶ Tempo di ricerca proporzionale alla dimensione
 - ▶ Lista ordinata per fare ricerca binaria, ma costi di gestione
- **Hash Table** – lista lineare con struttura dati ad hash table
 - Abbassa i tempi di ricerca su directory
 - **Collisioni** – situazioni dove due file name hanno hash sulla stessa locazione location
 - ▶ Metodo chained-overflow

Implementazione Directory

- ❑ **Hash Table** – lista lineare con struttura dati ad hash table
 - ❑ Chiave filename mappati su indici della tabella (funzione hash)
 - ❑ Abbassa i tempi di ricerca su directory
 - ❑ **Collisions** – situazioni dove due filename mappati sulla stessa locazione
 - ▶ Metodo chained-overflow

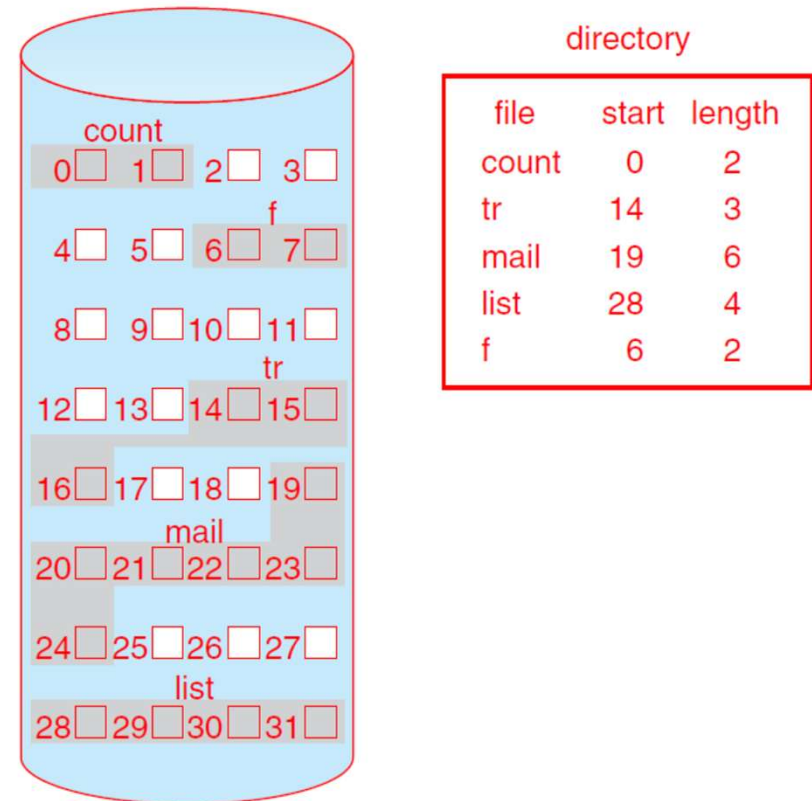


Metodi di Allocazione

- Lo storage ad accesso diretto permette di implementare diversi metodo di allocazione di file
 - Un metodo di allocazione è il modo in cui i blocchi sono allocati per i file
 - Criteri:
 - ▶ Spazio: memoria utilizzata in modo efficiente
 - ▶ Tempo: accesso veloce
- Tre metodi principali:
 - Allocazione contigua
 - Allocazione lincata
 - Allocazione indicizzata

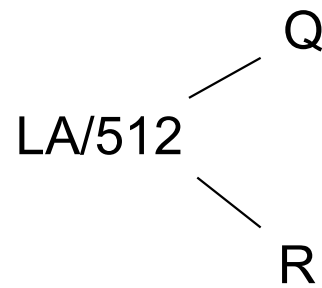
Metodi di Allocazione - Contigua

- **Allocazione Contigua** – ogni file occupa un insieme di blocchi contigui
 - Miglior performance in molti casi
 - Semplice
 - ▶ richiesta locazione di partenza (numero del blocco) e lunghezza (numero di blocchi)
 - Pro:
 - ▶ spostamenti della testina richiesti per accedere a tutti i blocchi di un file è trascurabile
 - ▶ Semplice anche l'accesso diretto
 - Contro:
 - ▶ Trovare spazio per il file (buchi)
 - ▶ Sapere a priori la dimensione del file
 - ▶ Frammentazione esterna
 - ▶ Necessità di compattare
 - off-line è downtime
 - on-line è lento
 - Allocazione contigua estesa
 - ▶ Aree contigue collegate (base, numero blocchi, base ext, numero blocchi ext)

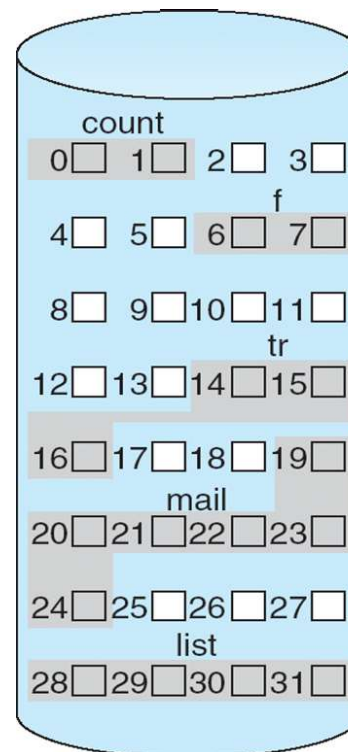


Allocazione Contigua

- Mapping da logica a blocchi
 - Logical Address/Block



Blocco da accedere = Q +
indirizzo di inizio
Dislocazione nel blocco = R



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Metodi di Allocazione - Contigua

- Pro: l'accesso ai file può essere eseguito semplicemente sia in modo sequenziale che diretto (meno veloce)
- Contro 1: è necessario trovare lo spazio libero per allocare il file
 - Diversi algoritmi
 - ▶ First-fit: primo buco abbastanza grande
 - ▶ Best-fit: il buco che approssima meglio la dimensione (per eccesso)
 - Si presenta il problema della frammentazione esterna
 - ▶ Può essere risolta tramite la compattazione
 - ▶ Si spostano i dati su un altro disco
 - ▶ Si crea una zona contigua di spazi libero
 - ▶ Si riportano i file su disco originale alloggiandoli in modo contiguo
 - ▶ La compattazione richiede molto tempo e può essere fatta
 - Off line: non è possibile usare il volume
 - On line: peggiora le prestazioni

Metodi di Allocazione - Contigua

- Pro: l'accesso ai file può essere eseguito semplicemente sia in modo sequenziale che diretto (meno veloce)
- Contro 2: è necessario stimare quanto spazio allocare ad un file
 - Quando un file finisce lo spazio disponibile
 - ▶ Il processo può essere interrotto
 - ▶ Il file può essere spostato in una zona di memoria più grande
 - Allocare più spazio di quello inizialmente necessario può portare alla frammentazione interna
 - ▶ Specialmente nel caso di file che crescono lentamente
- Alcuni sistemi usano l'allocazione contigua estesa
 - ▶ Quando un file necessita di spazio gli viene assegnata una seconda area contigua
 - ▶ Ogni area è caratterizzata da: blocco d'inizio, numero blocchi, area successiva
 - base, numero blocchi, base ext, numero blocchi ext

Sistema basato su Allocazione Estesa

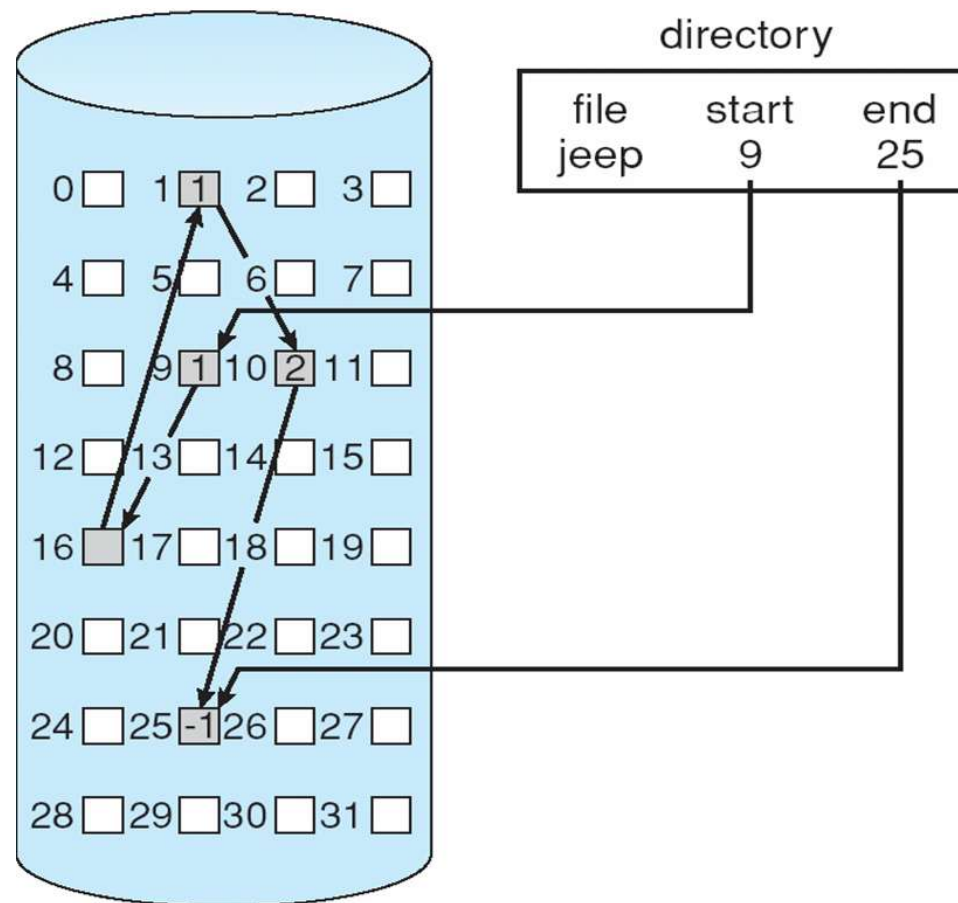
- File system più moderni (i.e., Veritas File System) usano uno schema di allocazione contiguo modificato
- Extent-based file system allocano blocchi di disco in locazioni contigue estese
- Un **extent** è un blocco contiguo di memoria
 - Extents sono allocati per allocazione di file
 - Un file consiste di uno o più estensioni
 - ▶ Prima viene allocato un chunk, poi vengono aggiunti altri
 - ▶ base, numero blocchi, base ext, numero blocchi ext

Metodi di Allocazione - Concatenata

- **Allocazione Concatenata** – ogni file è una lista linkata di blocchi
 - Ogni blocco contiene un pointer al prossimo blocco
 - ▶ File finisce al nil pointer
 - La directory memorizza per ogni file
 - ▶ Puntatore al primo e ultimo blocco

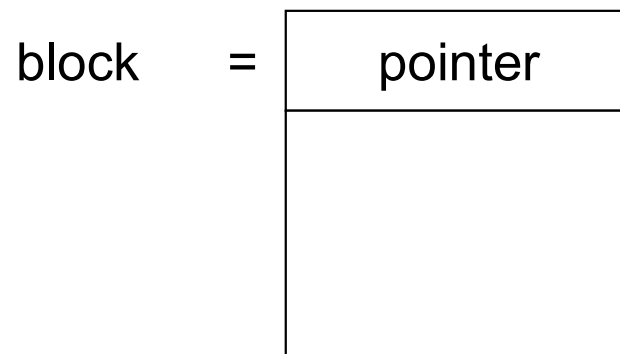
Il puntatore non è visibile all'utente, quindi lo spazio per l'utente è ridotto:

- Blocco da 512 Byte
- puntatore da 4 Byte
- Spazio utile: 508 Byte



Allocazione Concatenata

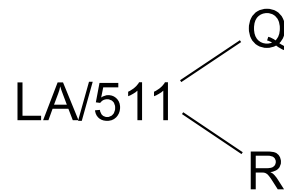
- Ogni file è una lista concatenata di blocchi del disco: i blocchi possono essere sparsi ovunque su disco



Il puntatore non è visibile all'utente, quindi "prende" spazio:

- Blocco da 512 Byte
- puntatore da 4 Byte
- Spazio utile: 508 Byte
- 0.78% disco usato da puntatore

Per accesso si fa riferimento al Q-esimo blocco nella lista concatenate che rappresenta il file, quindi si accede riferimento al record nel blocco



Allocazione Concatenata

- Creazione di un file:
 - Si aggiunge un elemento alla directory con Puntatore al primo blocco settato a null dimensione pari a 0

- Scrittura di un file:
 - Si cerca un blocco libero e si effettua la scrittura dei dati
 - Lo si concatena all'ultimo blocco del file (se presente)
 - Si aggiornano i puntatori della directory all'ultimo e al primo blocco (nel caso il file sia ancora vuoto)

- Lettura di un file:
 - Si scorrono in sequenza i blocchi seguendo la concatenazione

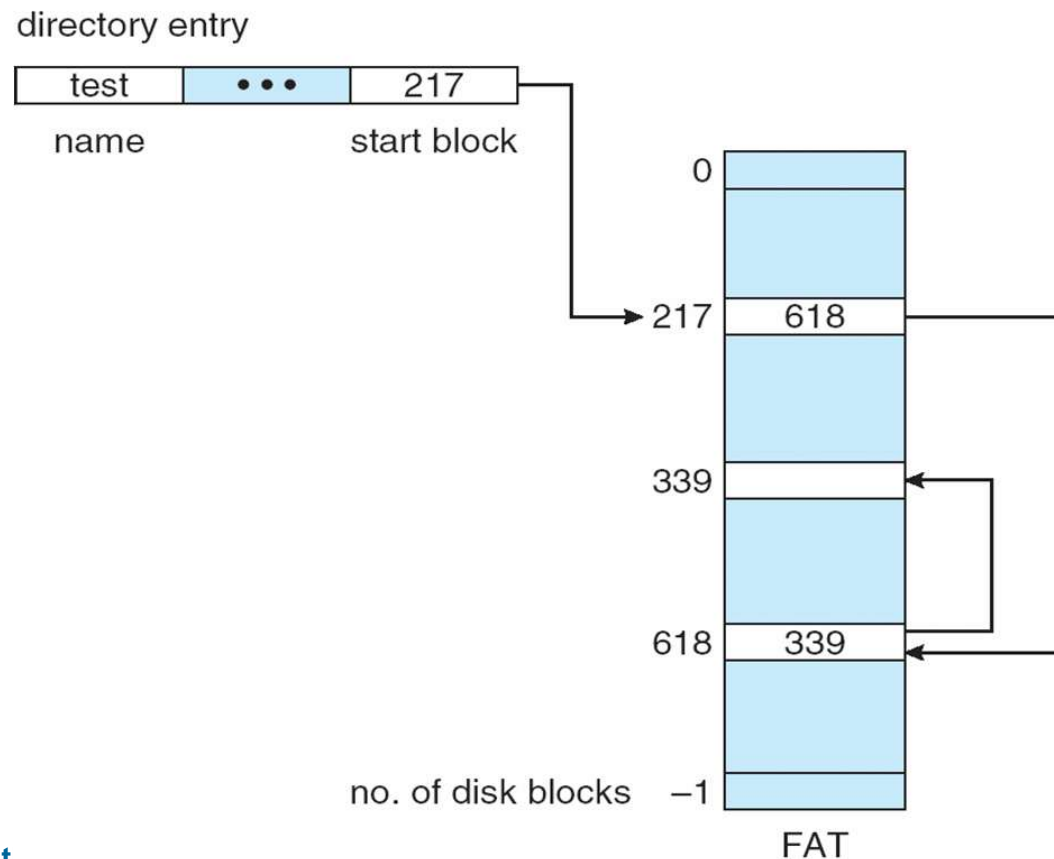
Metodi di Allocazione - Concatenata

- L'allocazione concatenata risolve i problemi dell'allocazione contigua:
 - Non c'è frammentazione esterna, non serve la compattazione
 - Non è necessario conoscere la dimensione del file al momento della creazione

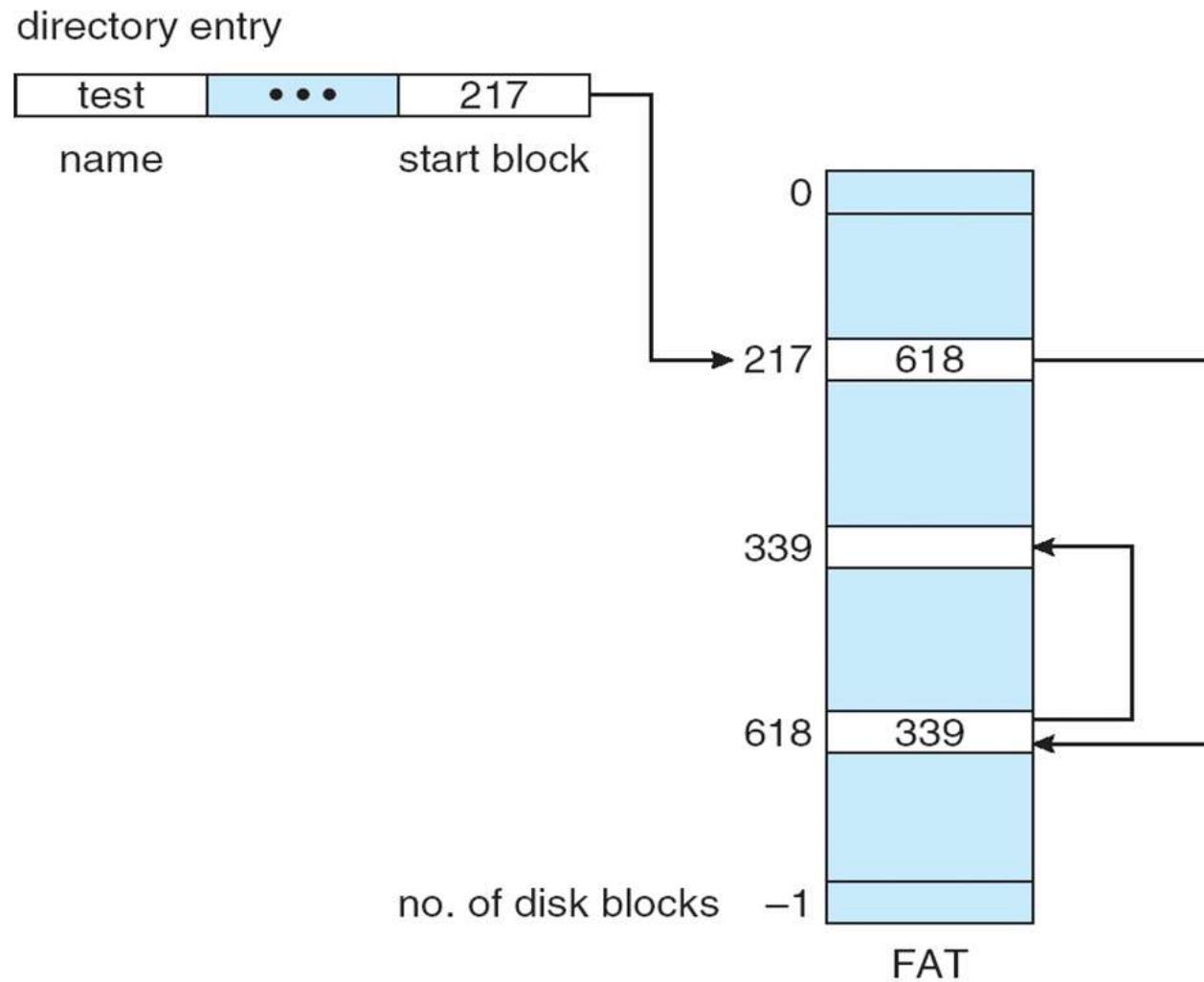
- Problemi:
 - È efficiente solo per l'accesso sequenziale (a causa dei puntatori)
 - Richiede più spostamenti della testina
 - I puntatori consumano spazio
 - ▶ Una soluzione è quella di allocare cluster di blocchi anziché singoli blocchi
 - ▶ Meno spazio sprecato per i puntatori, meno spostamenti della testina
 - ▶ Aumenta però la frammentazione interna
 - Perdita o danneggiamento di un puntatore
 - ▶ Potrebbe puntare ad un blocco vuoto o un blocco di un altro file
 - ▶ Possibile soluzione: liste doppiamente concatenate

File-Allocation Table

- Variante importante della lincata è FAT (File Allocation Table) - MSDOS
 - Nella prima parte del disco si istanzia un tabella
 - ▶ Contenente tanti elementi quanti sono i blocchi
 - ▶ Ordinata in base al numero del blocco
 - La directory contiene per ogni file il puntatore al suo primo elemento nella FAT
 - Seguendo i puntatori nella FAT si ricavano gli indici dei blocchi fisici di un file



File-Allocation Table

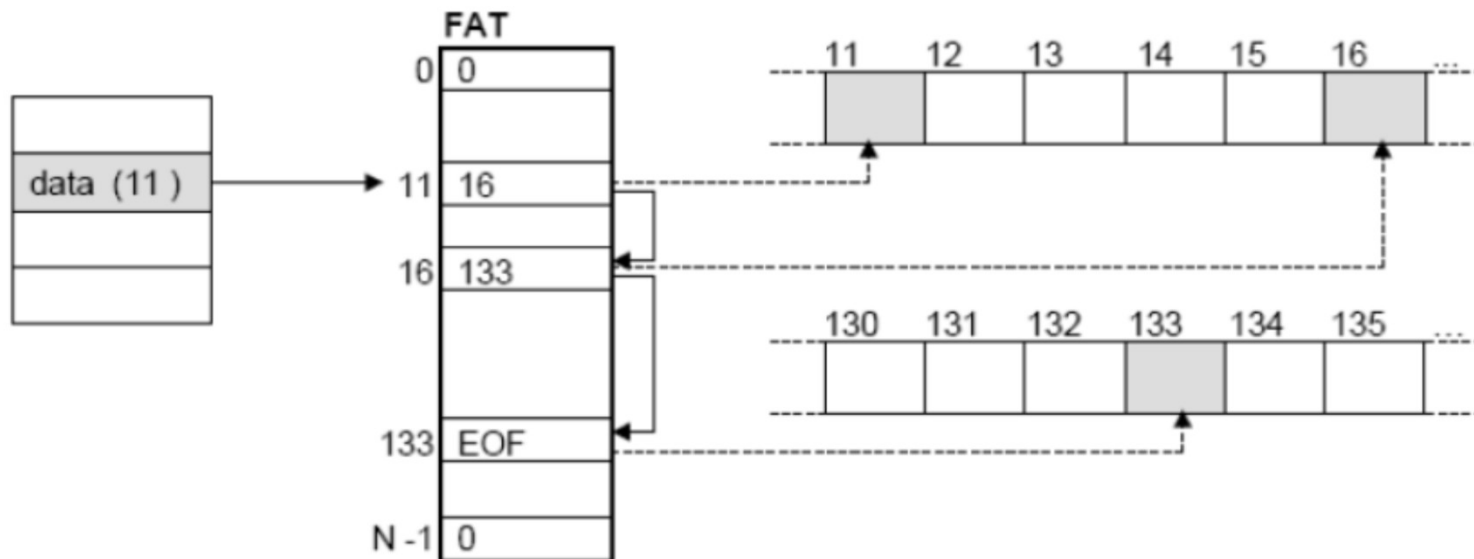


File-Allocation Table

- Variante importante è FAT (File Allocation Table)
 - Nella prima parte del disco si istanzia un tabella
 - ▶ Contenente tanti elementi quanti sono i blocchi
 - ▶ Ordinata in base al numero del blocco
 - La directory contiene per ogni file il puntatore al suo primo elemento nella FAT
 - ▶ Ogni elemento ha un numero di blocco ed è concatenato al blocco successivo nel file
 - ▶ L'ultimo elemento contiene un valore end of file
 - ▶ L'elemento di un blocco che non è associato a nessun file contiene uno 0
 - Seguendo i puntatori nella FAT si ricavano gli indici dei blocchi fisici di un file

File-Allocation Table

- Variante importante è FAT (File Allocation Table)



File-Allocation Table

- Variante importante è FAT (File Allocation Table)
- Per essere efficiente la FAT richiede l'uso di una cache
 - Altrimenti la testina del disco deve continuamente spostarsi tra l'inizio del disco e la locazione del blocco successivo
- Migliora le prestazioni nel caso di accesso diretto
 - La testina del disco scandisce la FAT e poi si fa uno spostamento grande per accedere al blocco voluto
- La differenza fra FAT12, FAT16 e FAT32 consiste in quanti bit sono allocati per numerare i blocchi del disco
 - Con 12 bit, il file system può indirizzare al massimo $2^{12} = 4096$ blocchi, mentre con 32 bit si possono gestire $2^{32} = 4.294.967.296$ blocchi
 - L'aumento del numero di bit di indirizzo dei blocchi e la dimensione stessa del blocco sono stati resi necessari per gestire unità a disco sempre più grandi e capienti

File-Allocation Table

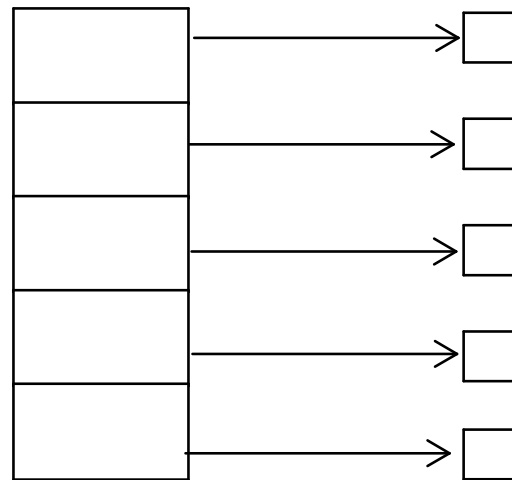
- ❑ Variante importante è FAT (File Allocation Table)
- ❑ Dimensione del blocco e FAT influenza la dimensione massima della partizione

Block size	FAT-12	FAT-16	FAT-32
0.5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

Metodi di Allocazione - Indicizzata

□ Allocazione indicizzata

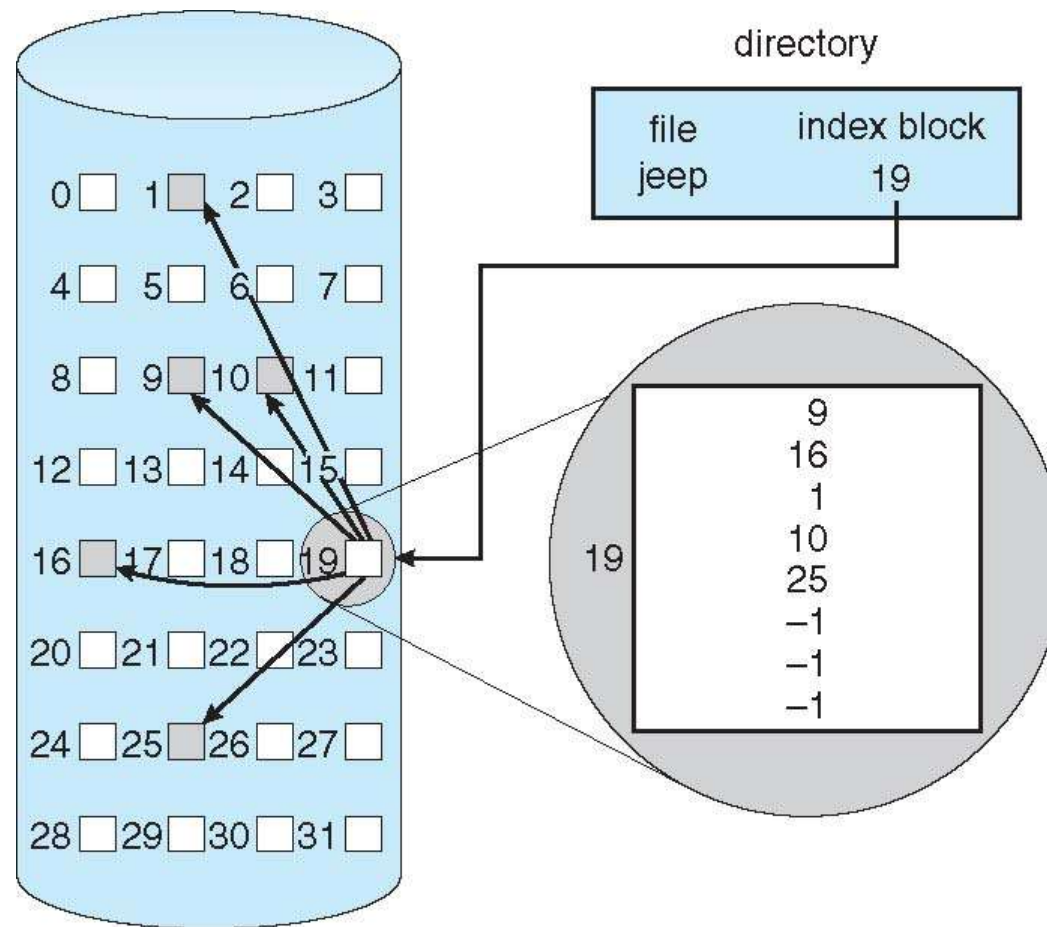
- Ogni file possiede un indice dei blocchi memorizzato in un blocco indice
 - ▶ Questo blocco contiene un array di puntatori agli altri blocchi del file
 - ▶ L'i-simo elemento dell'array punta all'i-simo blocco del file
- La directory memorizza per ogni file un puntatore al suo blocco indice
- Questo risolve il problema dell'accesso diretto presente nell'allocazione concatenata ed evita la frammentazione esterna



index table

Esempio di Allocazione Indicizzata

- Ogni file possiede un indice dei blocchi memorizzato in un blocco indice
 - Questo blocco contiene un array di puntatori agli altri blocchi del file
 - L'i-simo elemento dell'array punta all'i-simo blocco del file
- La directory memorizza per ogni file un puntatore al suo blocco indice



Allocazione Indicizzata

- Creazione di un file:
 - Si crea un blocco indice con tutti gli elementi settati a null
 - Si aggiunge un elemento alla directory con puntatore ad un blocco indice

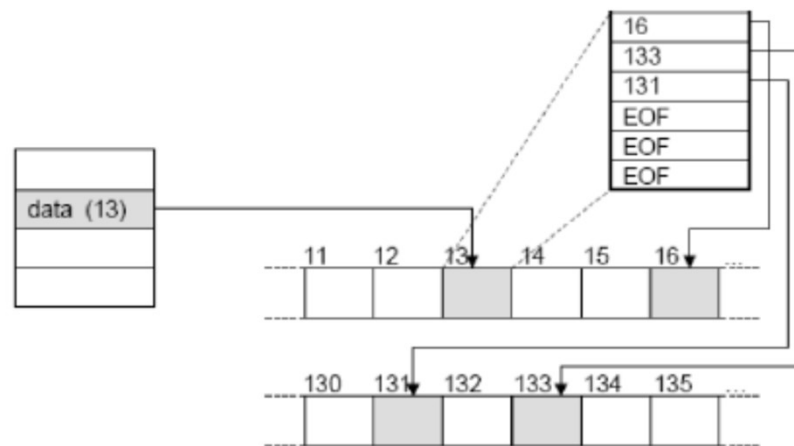
- Scrittura di un file:
 - Si cerca un blocco libero e si effettua la scrittura dei dati
 - Si aggiorna il blocco indice

- Lettura di un file:
 - si scorrono in sequenza i blocchi seguendo l'array dell'indice

Allocazione Indicizzata

❑ Problema:

- ❑ l'indice richiede l'uso di un intero blocco, quindi nel caso di un file piccolo si ha frammentazione interna

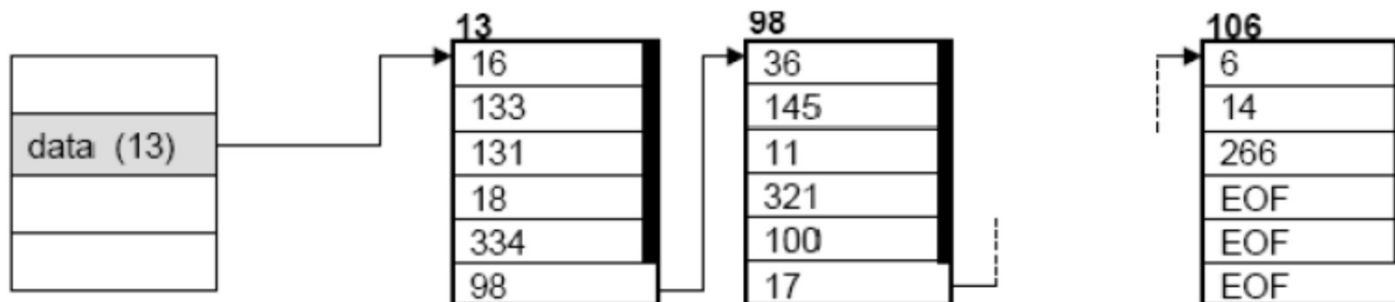


- ❑ Si dovrebbero usare blocchi piccoli in modo da ridurre la frammentazione
- ❑ Problemi con i file grandi
- ❑ Diverse soluzioni

Allocazione Indicizzata

□ Soluzioni:

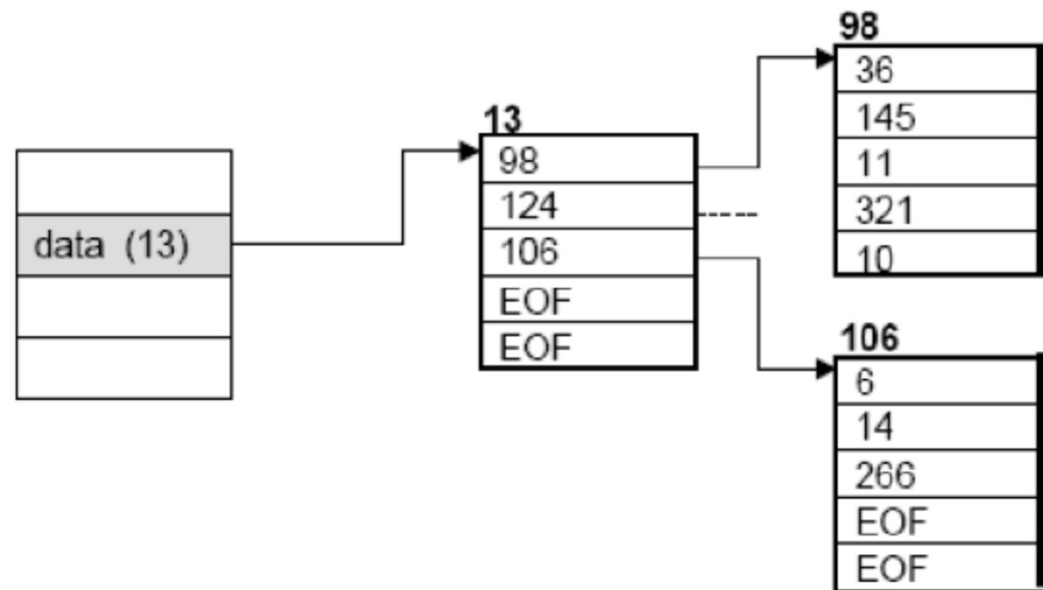
- Schema concatenato: si usano blocchi piccoli e nel caso di file grandi l'indice è composto da più blocchi concatenati



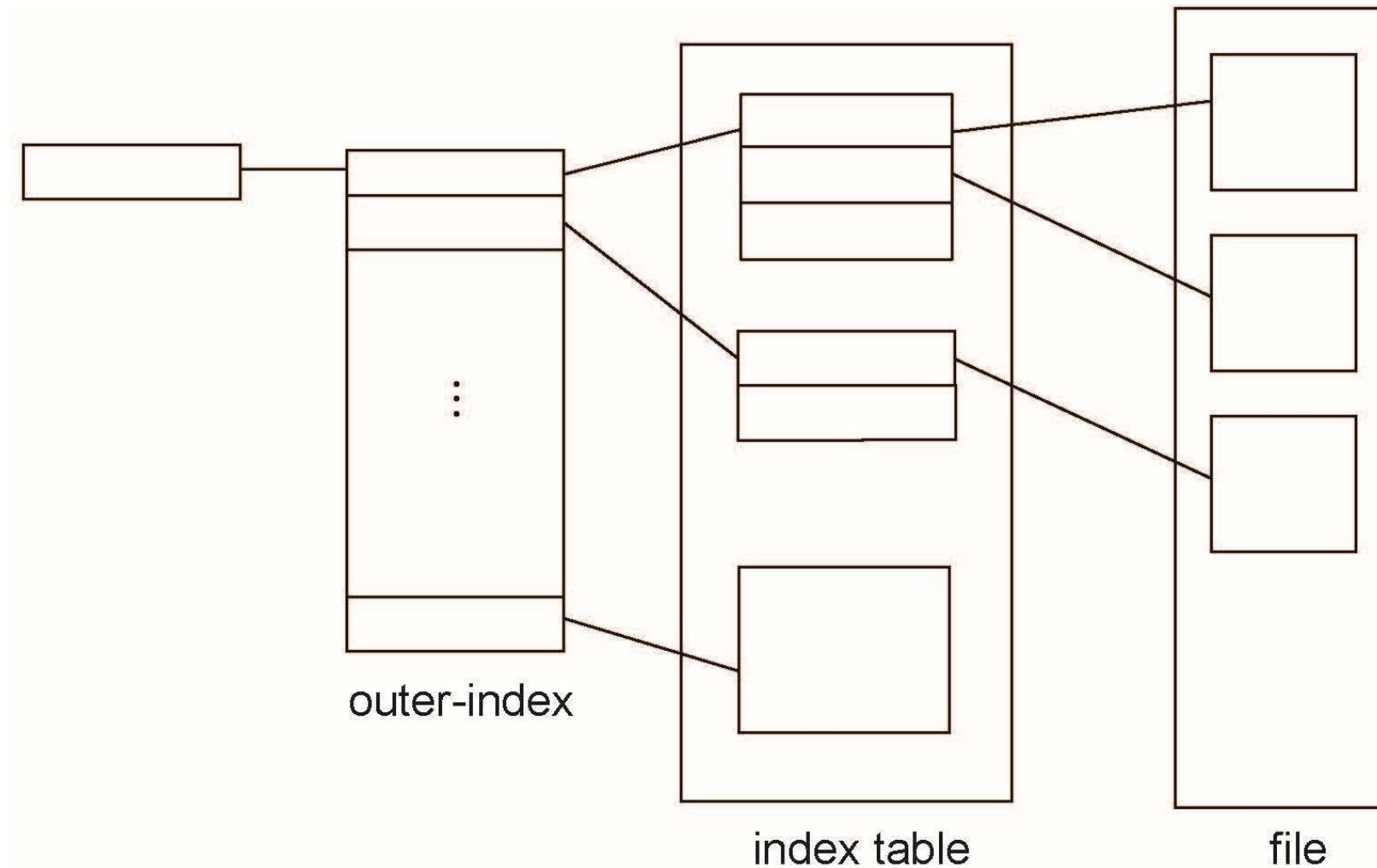
Allocazione Indicizzata

□ Soluzioni:

- Schema a più livelli: si ha un indice di primo livello che punta ad un insieme di indici di secondo livello, i quali puntano ai blocchi del file
- Con blocchi da 4 KByte e puntatori da 4 byte si hanno 1024 puntatori; con 2 livelli 2^{20} puntatori, quindi si possono indicizzare file da 4GB ($4 \times 2^{10} \times 2^{20}$)
- È possibile estendere lo schema a 3-4 livelli



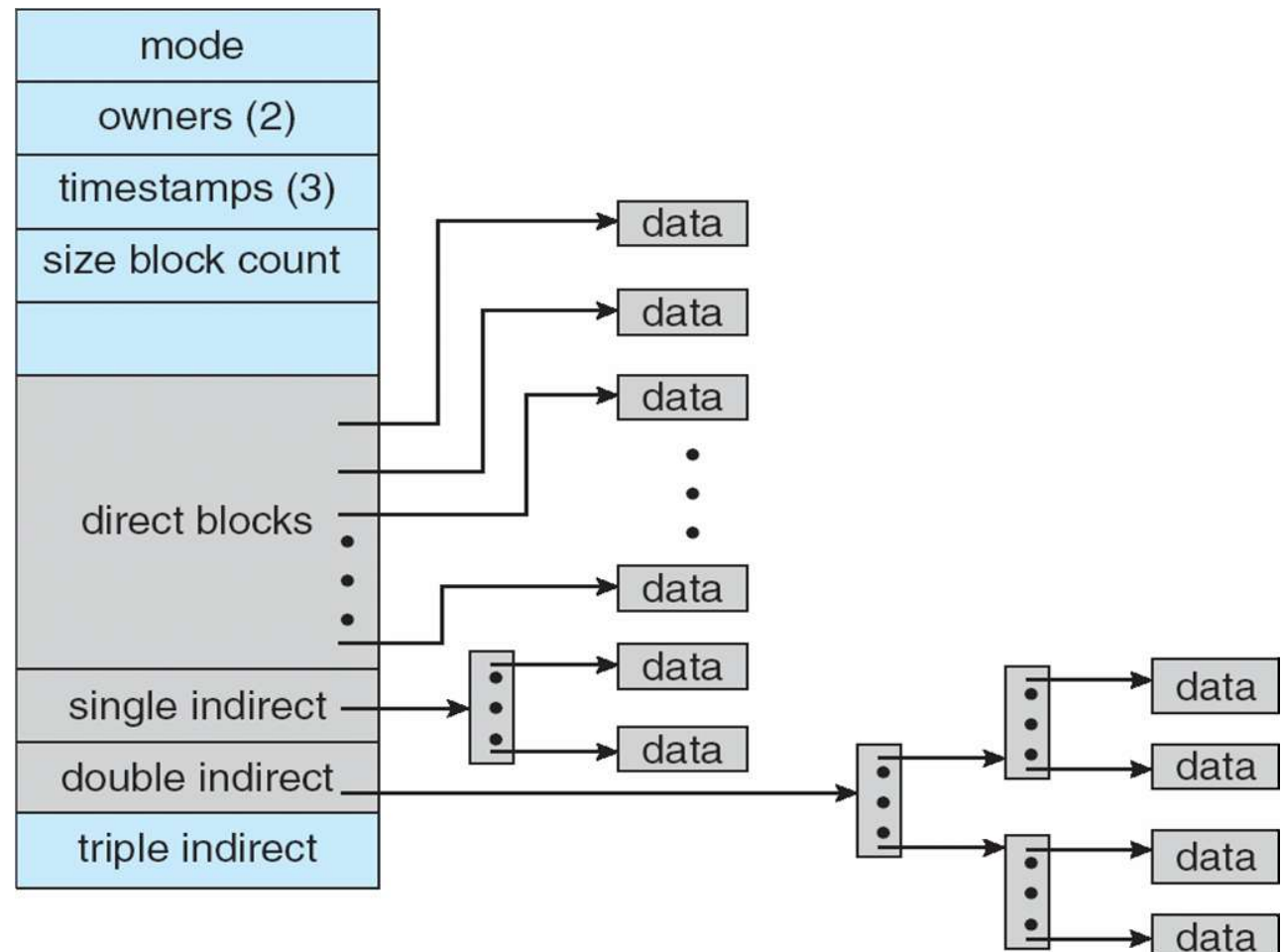
Allocazione indicizzata – Mapping



Schema Combinato: UNIX UFS

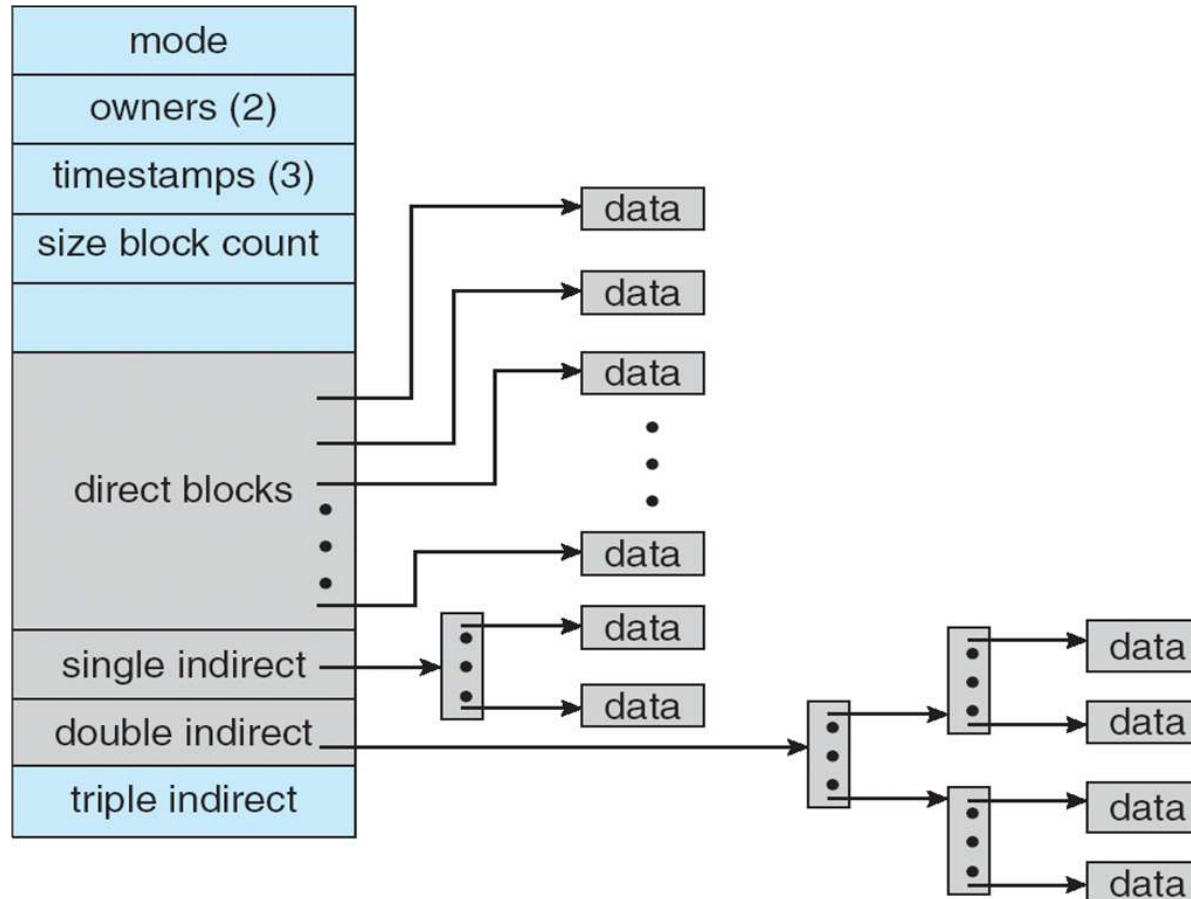
Schema combinato: usato nei sistemi Unix:

- Il FCB contiene 15 elementi per indicizzare i file
- I primi 12 sono puntatore diretti a dei blocchi
- Il 13° punta ad un blocco contenente un indice
- Il 14° punta ad un indice a due livelli
- Il 15° punta ad un indice a tre livelli



Schema Combinato: UNIX UFS

Assumendo indirizzi a 32-bit



Con questo metodo più blocchi indicizzati di quanti possono essere indirizzati con puntatori di file a 32-bit

Scelta Metodo Allocazione

- La scelta del metodo di allocazione deve considerare due fattori:
 - Efficienza di memorizzazione
 - Tempo di accesso ai dati

- La scelta è influenzata dalla modalità di accesso al file
 - L'allocazione contigua richiede un solo accesso al disco qualsiasi sia il tipo di accesso (sequenziale o diretto)
 - ▶ Anche con l'accesso diretto, noto l'indirizzo del primo blocco fisico e l'indirizzo del blocco logico desiderato, è possibile calcolare l'indirizzo fisico di accesso
 - L'allocazione concatenata invece è efficiente per l'accesso sequenziale, ma non per quello diretto
 - ▶ Accesso all'i-esimo blocco porta all'accesso di i blocchi intermedi

Scelta Metodo Allocazione

- Per questo motivo alcuni sistemi gestiscono
 - File ad accesso sequenziale con allocazione concatenata
 - File ad accesso random con l'allocazione contigua
 - ▶ È necessario specificare l'uso del file a tempo di creazione
 - ▶ Si dichiara anche la dimensione massima del file
 - ▶ È sempre possibile convertire il formato del file
 - Le prestazioni dell'allocazione indicizzata invece dipendono
 - ▶ Dalla profondità dell'indice (quanti blocchi indice)
 - ▶ Dalla dimensione del file (se grande occorre scorrere molti blocchi indice)
 - ▶ Occorre in memoria principale il blocco indice per essere efficienti
- Alcuni sistemi usano
 - ▶ Allocazione contigua per file piccoli (più comuni)
 - ▶ Allocazione indicizzata per file grandi
- Altre ottimizzazioni per ridurre i tempi di seek, ma solo per HDD, non per NVM

Gestione dello Spazio Libero

- Trovare in modo veloce un insieme di blocchi da allocare ad un file
 - Tener traccia dei blocchi non allocati
 - Lista dello spazio libero (non necessariamente implementata come lista)

- Creazione di un file:
 - Si cerca nella lista il numero di blocchi necessari
 - Si allocano al file
 - Si rimuovono dalla lista

- Eliminazione di un file
 - I blocchi associati al file vengono deallocati
 - Si aggiungono alla lista

Gestione dello Spazio Libero

- Diverse possibili implementazioni:
 - Vettore di bit
 - Lista concatenata
 - Raggruppamento
 - Conteggio

Vettore di Bit o BitMap

- Si memorizza un array di lunghezza pari al numero dei blocchi
- Il valore delle celle indica la disponibilità di un blocco
 - 1 libera
 - 0 occupata
- Per creare un file
 - Ricerca del primo bit a 1 (allocazione concatenata o indicizzata)
 - Ricerca di un blocco di bit a 1 sufficientemente grande (allocazione contigua)
- Pro: semplice ed efficiente
- Contro:
 - Per essere efficiente deve però essere mantenuta in memoria centrale
 - Dischi grandi richiedono vettori di bit molto grandi

Vettore di Bit

- Bit map richiede spazio extra

- Esempi:

disk size = 1.3 GB

block size = 512 byte

bitmap = 332 KB

se cluster di 4 -> 83 KB

block size = 4KB = 2^{12} bytes

disk size = 2^{40} bytes (1 terabyte)

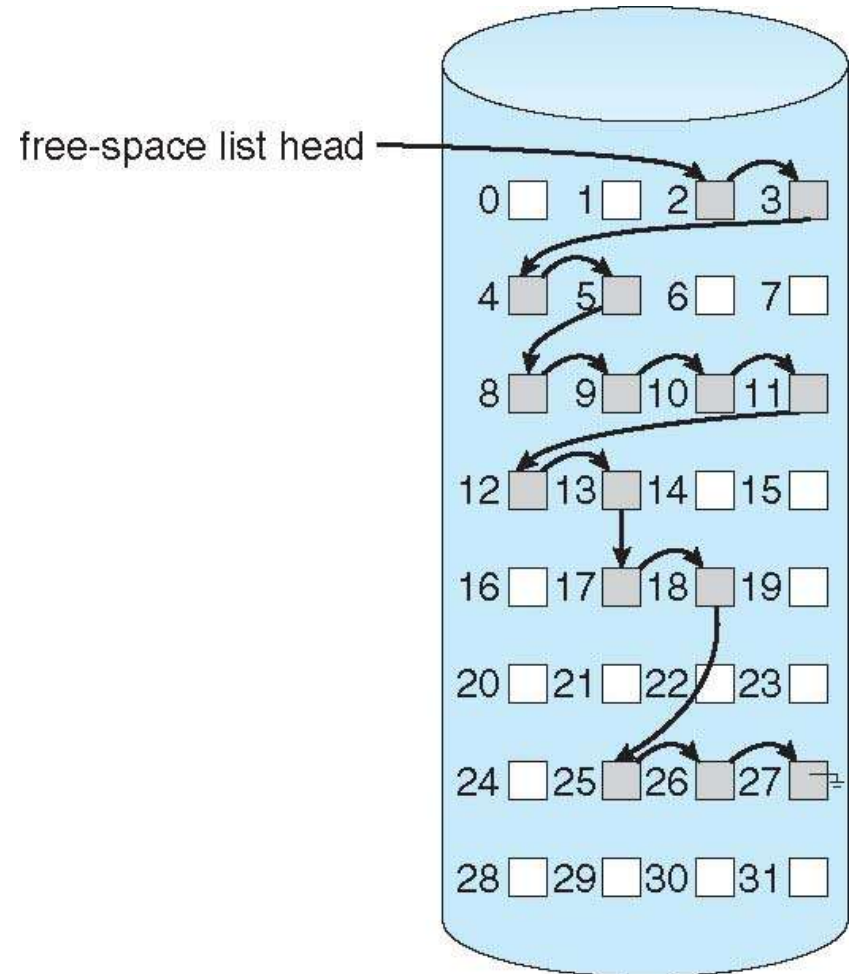
$n = 2^{40}/2^{12} = 2^{28}$ bits (o 32MB)

se cluster di 4 blocchi -> 8MB of memory

- Dischi aumentano di dimensioni quindi aumenta il problema

Lista Concatenata

- ❑ Si memorizza in memoria centrale un puntatore al primo blocco libero
- ❑ Ogni blocco libero viene poi concatenato al successivo
- ❑ Contro: è poco efficiente in quanto scorrere l'intera lista richiede molti accessi alla memoria
- ❑ Fortunatamente lo scorrimento della lista è un evento raro
- ❑ Il SO si limita a cercare il primo blocco libero per allocarlo ad un file
- ❑ Allocated il blocco si aggiorna il puntatore al primo blocco libero in memoria centrale
- ❑ FAT contiene link a blocchi liberi



Raggruppamento e Conteggio

□ Raggruppamento

- Modifica dell'approccio a free-list
- Il primo blocco contiene gli indirizzi di n blocchi liberi
 - ▶ Al primo accesso subito info sui blocchi liberi
- n-1 sono effettivamente liberi
- L'ultimo contiene a sua volta n indirizzi di blocchi liberi

□ Conteggio

- Sfrutta il fatto che tipicamente si ha più di un blocco contiguo libero
- Si usa un array in cui ogni elemento contiene
 - ▶ Indirizzo del primo blocco libero
 - ▶ Conteggio degli n blocchi contigui liberi

Linux File System

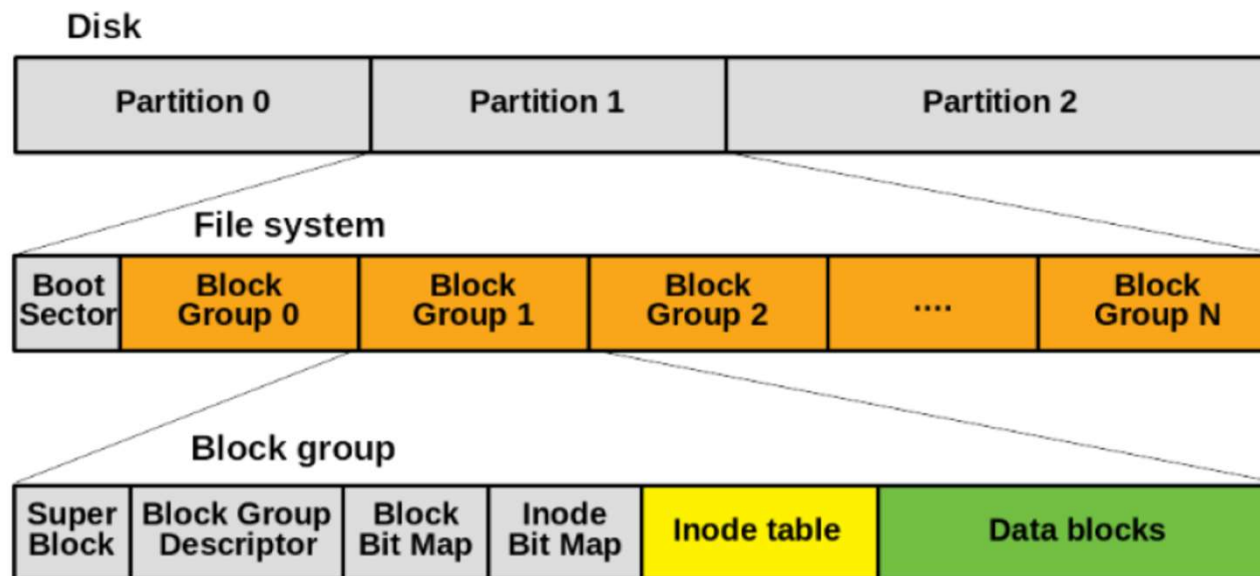
□ Evoluzione

- Il file system EXT (Extended) sviluppato da Rémy Card e rilasciato in Linux nel 1992 per superare le limitazioni di Minix filesystem
 - ▶ metadati basati su Unix filesystem (UFS) o Berkeley Fast File System (FFS)
 - ▶ Superato rapidamente da EXT2 filesystem
- Come Minix, EXT2 ha un boot sector nel primo settore del HDD su cui è installato
 - ▶ Ha uno small boot record e una partition table
- EXT3 ha il journal che dichiara le modifiche sul FS
 - ▶ Per il resto è simile ad EXT2
- EXT4 filesystem migliora le performance, leggibilità e capacità
 - ▶ FS di dimensioni fino a 1 exbibyte e file di dimensioni fino a 16 tebibyte
 - ▶ I file di grandi dimensioni vengono suddivisi in più estensioni
 - ▶ Allocazione multiblocco
 - ▶ Meno frammentazione
 - ▶ Dimensione di inode più grande (256 byte)

Linux ext File System

- **ext3, ext4** standard per il disk file system in Linux
 - Simile a BSD Fast File System (FFS) per trovare i blocchi di un file
 - Supera **extfs, ext2**

Ext2 on a disk consists of many **groups of disk blocks** (of the same size, located sequentially one after the other). Block groups reduce **file fragmentation**.



Source: <https://computing.ece.vt.edu/~changwoo/ECE-LKP-2019F/I/lec21-fs.pdf>

File System NTFS

- ❑ New Technology File System (sostituisce FAT)
- ❑ Struttura fondamentale di NTFS è un *volume* (in una partizione)
- ❑ NTFS usa i *cluster come unità di allocazione del disco*
 - ❑ Numero di settori con potenza di 2
 - ❑ NTFS usa logical cluster numbers (LCNs) come disk addresses
- ❑ File NTFS è un oggetto strutturato (non flusso di byte)
- ❑ Name space NTFS organizzato in una gerarchia di directory

❑ Metadati

- ❑ Master File Table (MFT)
 - ▶ Struttura del file system
 - ▶ Una o più entry per ogni file
- ❑ Log file per journaling
- ❑ Informazioni sul volume
- ❑ Bitmap

MFT
MFT copy (partial)
Log file
Volume file
Attribute def. table
Root directory
Bitmap file
Boot file
Bad cluster file
...
User files and dirs.

Efficienza e Prestazioni

- I dischi sono tipicamente conosciuti come il collo di bottiglia di ogni sistema
 - A causa della loro velocità di accesso
 - Anche NVM sono lente rispetto a CPU

- Gli algoritmi di allocazione dei blocchi e gestione delle directory devono essere scelti con cura al fine di ottimizzare
 - Efficienza nell'uso dei dischi
 - Prestazioni

Efficienza

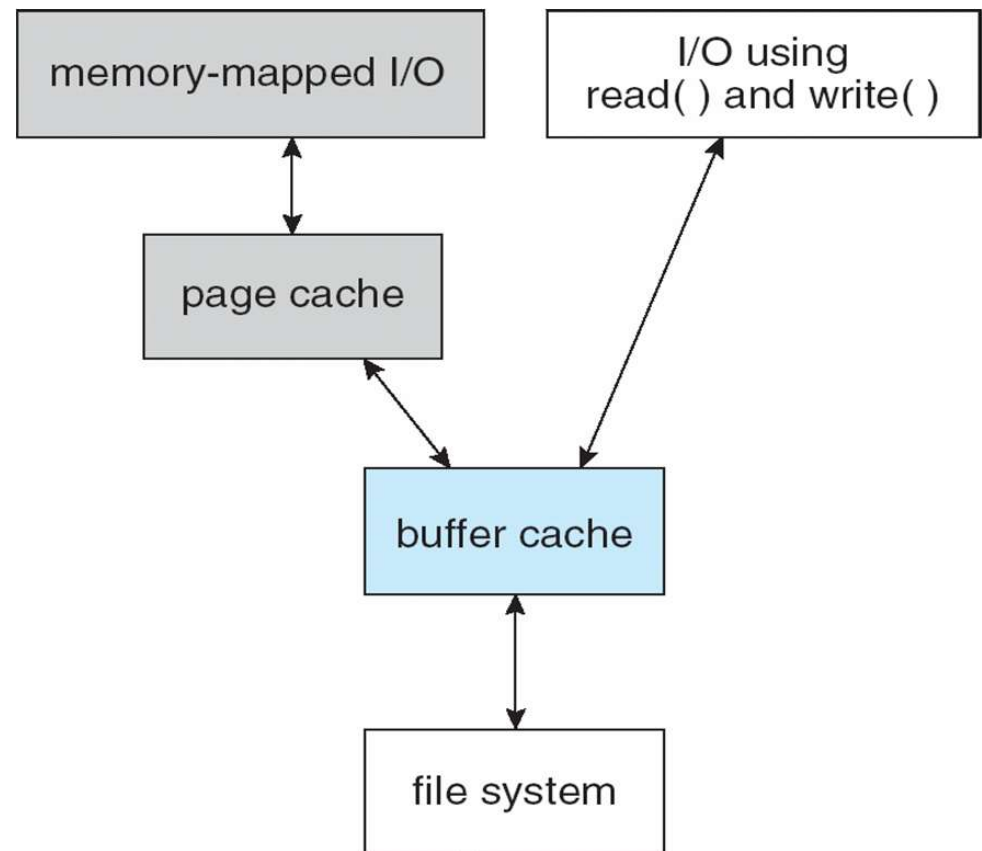
- Diverse scelte influenzano l'efficienza del disco
 - Allocazione dei FCB:
 - ▶ alcuni SO, es. Unix, allocano preventivamente i FCB e li distribuiscono nel file system
 - ▶ Anche un disco senza file ha una certa quantità di spazio occupata dai FCB
 - ▶ Questa scelta può migliorare le prestazioni del file system
 - Scelta degli attributi da memorizzare per ogni file:
 - ▶ Salvare le date di accesso ai file può essere utile per fornire informazioni all'utente
 - ▶ Tuttavia questo richiede due accessi al disco ogni volta che si legge un file
 - ▶ Uno in lettura e uno in scrittura per modificare la directory
 - ▶ Gli attributi da memorizzare vanno scelti con cura
 - Lunghezza dei puntatori per accedere ai dati:
 - ▶ Più bit si usano per i puntatori più grandi possono essere i file (32 è bit $2^{32}=4\text{GB}$)
 - ▶ Tuttavia puntatori grandi richiedono più spazio per eseguire gli algoritmi di allocazione e gestione dello spazio libero (è spazio non usabile dall'utente)
 - ▶ Es. IBM PC XT con FAT entry di 12 bit, quindi 2^{12} link di cluster di 8 KB, cioè 32 MB, dunque partizioni fino a 32 MB – FAT espansa prima a 16 bit poi a 32 bit

Prestazioni

- Una volta scelti gli algoritmi per la gestione del file system si possono adottare diverse tecniche per migliorare le prestazioni
- **Cache del disco** (buffer cache): area riservata della memoria centrale dove vengono mantenuti i blocchi usati di recente
 - ... che probabilmente verranno riusati a breve
 - memoria fisica (non virtualizzata) riservata al Kernel
- **Cache delle pagine**: area riservata della memoria centrale (kernel) per la gestione ottimizzata dell'accesso ai file tramite pagine
 - memoria virtuale non solo per la gestione dei processi, ma anche per l'accesso ai file (più veloce)
 - Utile per gestire in modo efficiente l'accesso ai file (memoria virtuale unificata)

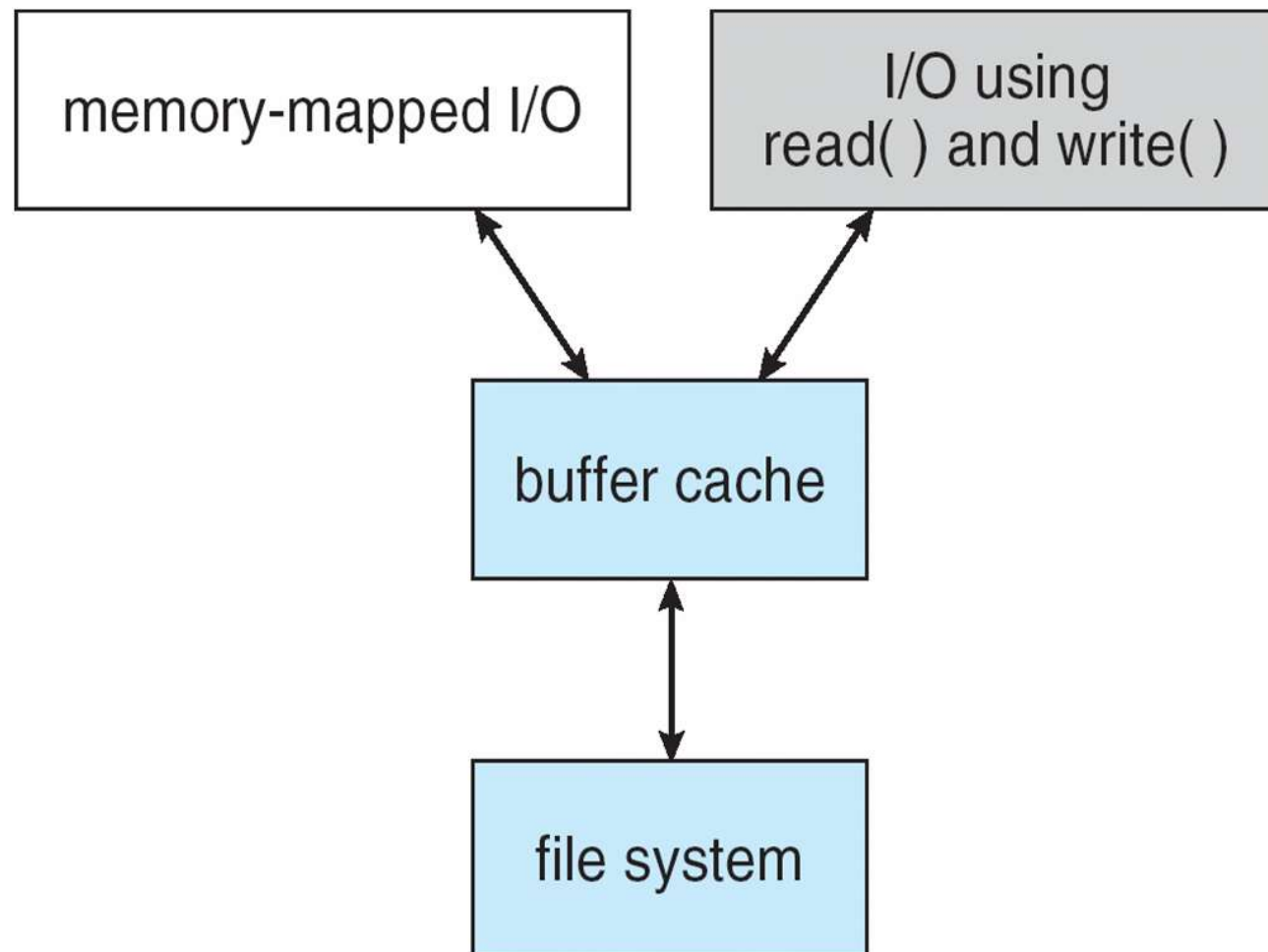
Page Cache vs Buffer Cache

- ❑ Accesso in memoria sia tramite le chiamate di sistema `read()` e `write()` sia con file mapping
- ❑ Se non si usa la cache unificata
- ❑ Le chiamate di sistema usano direttamente la buffer cache (FS cache)
- ❑ Il sistema di memoria virtuale non può interfacciarsi direttamente con la buffer cache
- ❑ È necessario copiare nella cache delle pagine il contenuto del file presente nella buffer cache
- ❑ Fenomeno del double caching: doppio passaggio di cache
- ❑ Spreco di memoria, CPU e I/O
- ❑ Possibili inconsistenze



Buffer Cache Unificata

- Con la FS cache unificata il problema del double caching viene risolto
- Il sistema di memoria virtuale può interfacciarsi direttamente con la cache del file system



Sostituzione Pagine

- Algoritmo di sostituzione della pagine:
 - Il metodo di accesso al file influenza la scelta dell'algoritmo di sostituzione delle pagine
 - In caso di accesso sequenziale LRU non è ottimale
 - ▶ la pagina più recente non è quella che sarà usata prima
 - Rilascio indietro:
 - ▶ Rimossa la pagina al riferimento alla pagina successiva (voltare pagina)
 - Lettura anticipata:
 - ▶ Si copiano nella cache la pagina richiesta le successive
 - Utilizzando queste tecniche si ha un notevole risparmio di tempo

Scritture

- Scritture sincrone e asincrone:
 - Influenzano le prestazioni di I/O
- Scrittura sincrona: le scritture su disco avvengono nell'ordine in cui il driver riceve le richieste
 - Il chiamante è bloccato fino a quando il dato non viene scritto
- Scritture asincrone: le scritture avvengono sulla cache del disco
 - Il driver del dispositivo si occupa poi di trasferire i dati sul disco
 - Il chiamante non resta bloccato
- Le scritture asincrone sono usate più frequentemente
 - Per i metadati le scritture possono essere sincrone per evitare problemi di consistenza

Recupero

- ❑ Operazioni eseguite molto frequentemente, come la creazione di un file, comportano molti cambiamenti nelle strutture dati del file system
 - ❑ Struttura della directory
 - ❑ Lista dello spazio libero
 - ❑ FCB
- ❑ Se un crollo del sistema avviene prima che tutte queste operazioni siano completate possono crearsi delle incoerenze
- ❑ Es., un nuovo FCB potrebbe essere stato allocato ma la struttura delle directory potrebbe non contenere il puntatore ad esso
- ❑ Il SO adotta diverse tecniche per risolvere le incoerenze

Controllo di Consistenza

- Per scoprire eventuali errori è necessario analizzare i meta-dati
- Questo richiede molto tempo
 - Si analizzano solo i cambiamenti
 - Prima di modificare i meta-dati il SO mette un bit di modifica a 1
 - Se le modifiche terminano con successo il bit viene rimesso a 0
 - Se resta ad un 1 significa che c'è stato un crollo e al riavvio viene eseguito il verificatore della coerenza
- Consistency checker confronta i dati nelle directory e altri metadati con i dati memorizzati e prova a correggere:
 - Con l'allocazione concatenata i puntatori permettono di ricostruire un file
 - ▶ Si possono ricreare i file in una directory
 - Con allocazione indicizzazione grave la perdita della dir che punta all'indice
 - ▶ I blocchi infatti non contengono informazioni sugli altri blocchi del file
 - ▶ In scrittura si aggiornano le directory in modo sincrono (non sempre risolve)

Registrazione Modifiche (log)

- La verifica della coerenza comporta alcuni problemi
 - Non sempre le incoerenze sono risolvibili
 - In alcuni casi la risoluzione richiede l'intervento umano
 - Richiede molto tempo

- Si adottano degli algoritmi usati nelle basi di dati
 - Si mantiene un log dove salvare in modo sequenziale le modifiche ai metadati
 - ▶ [log-based transaction-oriented file systems \(journaling\)](#)
 - Quando si opera su un file l'operazione è considerata committed quando
 - ▶ Tutte le modifiche da apportare ai meta-dati sono state salvate nel log (transazione)
 - La chiamata di sistema restituisce il controllo all'utente
 - SO si occupa di aggiornare in modo asincrono i meta-dati come indicato dal log
 - Quando tutti i metadati sono stati aggiornati le modifiche vengono tolte dal log
 - ▶ Implementato come un buffer circolare (sovrascrive i vecchi valori)
 - Usato NTFS, Veritas, UFS su Solaris, ext3, ext4, ZFS

Registrazione Modifiche (log)

- Se il sistema crolla, al momento del riavvio
 - Si analizza il log e si eseguono tutte le transazioni registrate
 - queste modifiche non erano ancora state fatte sui metadati
- Problema quando la transazione non è committed al momento del crash
 - Tutti i cambiamenti fatti al FS annullati per mantenere consistenza
 - ▶ unico ripristino necessario
- Il log è tipicamente salvato in un'area dedicata del disco
 - Accedere in modo sequenziale e sincrono al log per salvare le transazioni
 - ▶ È più veloce che accedere in modo diretto e sincrono ai blocchi contenenti i meta-dati
- Questa tecnica diminuisce il tempo in cui un processo resta bloccato in attesa dell'aggiornamento dei metadati

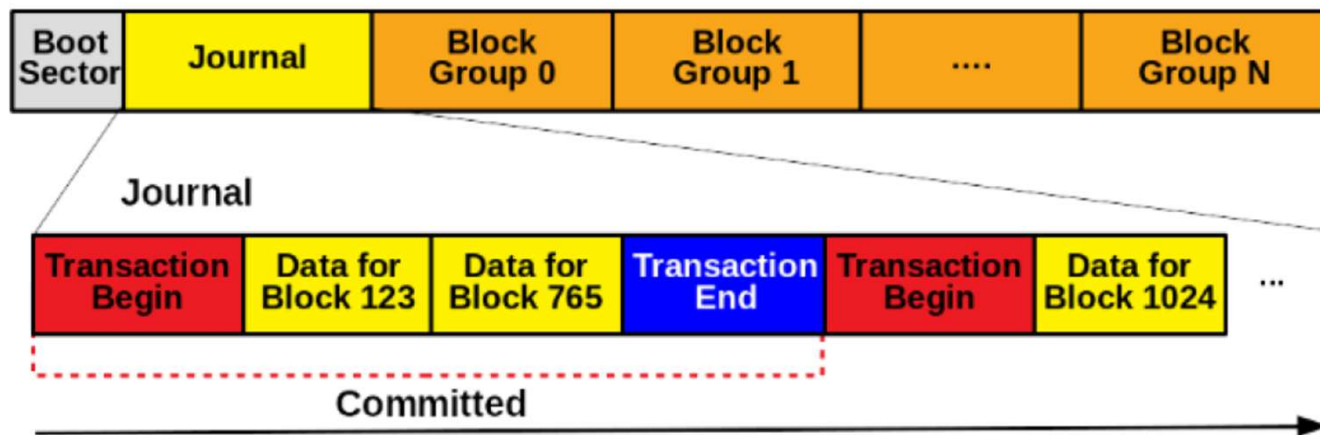
Journaling (Linux)

- ❓ ext3/ext4 implements **journaling**, with file system updates first written to a log file in the form of **transactions**
 - ❓ Once in log file, considered committed
 - ❓ Over time, log file transactions replayed over file system to put changes in place

Journal is logically a **fixed-size, circular array**.

- Implemented as a **special file** with a **hard-coded inode** number.
- Each journal **transaction** is composed of a **begin** marker, **log**, and **end** marker.

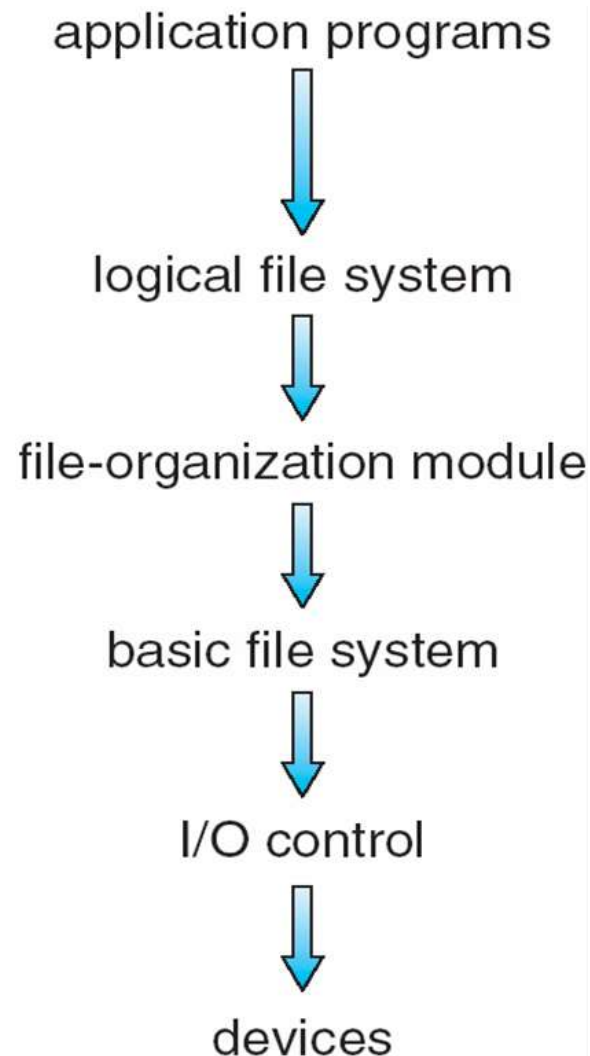
File system



Source: <https://computing.ece.vt.edu/~changwoo/ECE-LKP-2019F/I/lec21-fs.pdf>

Struttura del File-System

- Il file system sono tipicamente realizzati secondo una struttura a strati
- Il livello superiore si basa sui servizi offerti dal livello inferiore



Strati del File System

- Il file system sono tipicamente realizzati secondo una struttura a strati
- Il livello superiore si basa sui servizi offerti dal livello inferiore
- Livelli di un file system:
 - **Controllo dell'I/O:** driver dei dispositivi e gestore degli interrupt
 - ▶ Si occupa del trasferimento dell'informazione dalla memoria di massa a quella centrale
 - **File System di base:** invia dei comandi di base ai device driver per scrivere e leggere blocchi fisici, gestisce buffer e cache
 - **Modulo di organizzazione dei file:** conosce file e struttura a blocchi logici
 - ▶ Comprende anche il gestore dello spazio libero (mantiene la lista dei blocchi liberi)
 - **File System Logico:** gestisce i metadati e la struttura delle directory
 - ▶ Gli attributi dei file vengono salvati in File Control Blocks (FCB)

Strati del File System

- **Device driver** gestiscono i dispositivi I/O al livello di controllo I/O

- **Basic file system (block I/O subsystem)**
 - comandi del tipo “prendi il blocco 123” li traduce in comandi generici per i device driver
 - Schedulazione delle richieste I/O con indirizzi logici
 - Gestisce anche buffer e cache (allocazione, freeing, replacement)
 - ▶ Buffer mantengono data in transito, cache mantengono metadati frequentemente usati

- **File organization module** livello di gestione dei file, indirizzi logici e blocchi fisici
 - Ogni blocco è numerato da 0 a N
 - Gestisce lo spazio libero e l’allocazione del disco

Strati del File System

- **Logical File System** gestisce informazione sui metadati
 - Traduce i nomi di file in file number, file handle, locazioni mantenendo i File Control Blocks (**inodes** in UNIX)
 - Gestione delle directory
 - Protezione

- Stratificazione utile per ridurre la complessità e la ridondanza,
 - Il codice è più riutilizzabile
 - ▶ Diversi file system possono essere implementati nello stesso SO riusando gli strati di basso livello
 - Ma aggiunge overhead e può ridurre la performance

- La decisione di quanti strati implementare è importante nel progetto del SO

Strati del File System

- Esistono diversi file system, gli SO ne supportano più di uno
 - Ognuno con il suo formato
 - CD-ROM è ISO 9660;
 - Unix ha **UFS**, FFS;
 - Windows ha FAT, FAT32, NTFS come floppy, CD, DVD Blu-ray,
 - Linux ha più di 40 tipi, con **extended file system** ext2, ext3, ext4 principali; più distributed file systems, etc.
 - Nuovi ancora in arrivo – ZFS, GoogleFS, Oracle ASM, FUSE

Implementazione del File-System

- Un file system richiede la definizione di diverse strutture dati
- Alcune risiedono in memoria di massa, altre in memoria centrale
- Strutture dati in memoria di massa:
 - **Boot Control Block:** (per volume) contiene informazioni necessarie per avviare il SO contenuto nel disco da quel volume (se il volume non contiene SO è vuoto)
 - **Volume Control Block:** (per volume) contiene dettagli riguardanti il volume
 - ▶ Numero di blocchi fisici e dimensione
 - ▶ Contatore dei blocchi liberi e puntatori ai blocchi liberi
 - ▶ Contatore dei FCB liberi presenti nel volume e puntatori ad essi
 - ▶ UFS: superblock, NTFS: master file table
 - **Struttura della directory:** (per FS) organizza i file
 - ▶ UFS: inode, filename, NTFS: master file table
 - **File Control Block:** (per file) attributi dei file
 - ▶ inode number, permessi, dimensione, date
 - ▶ UFS: inode, NTFS: in master file table

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Strutture File System in Memoria

- Dati in memoria per la gestione del FS e per aumentarne la performance
 - Dati acquisiti quando il FS è montato
 - Dai aggiornati durante l'utilizzo
 - Dati cancellati quando è smontato

- Strutture dati in memoria centrale:
 - **Tabella di montaggio:**
 - ▶ Informazioni relative ai volumi attualmente montati
 - **Struttura delle directory** (in parte):
 - ▶ Cache dei dati su directory recentemente usate dai processi
 - **Tabella dei file aperti** (livello SO):
 - ▶ Copia del FCB più dati di accesso per ogni file aperto
 - **Tabelle dei file aperti** (livello processo):
 - ▶ puntatori alla entry nella tabella di sistema dei file aperti più altre info
 - **Buffer di Blocchi:**
 - ▶ Buffer per tenere i blocchi letti e scritti in memoria

Creazione file

- Per creare un nuovo file un processo utilizza il FS logico
 - Chiamato il Logical File System (che conosce il formato delle directory)
 - LFS genera e alloca un nuovo FCB
 - LFS crea e alloca un nuovo FCB
 - LFS legge la directory nella memoria centrale
 - LFS Aggiorna la directory in memoria centrale aggiorna con nuovo file
 - LFS Riscrive la directory in memoria di massa
- Per alcuni SO la directory è un file (UNIX) per altri strutture diverse (Windows)
- Il Logical File System chiama il File Organization Module che chiama il Basic File System

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Apertura e Chiusura File

- Open() e Close()
- La system call passa il nome del file al Logical File System
- LFS controlla nella tabella dei file aperti (sistema) se il file è già aperto
 - In caso negativo
 - ▶ Cerca il nome del file nella directory (usando cache per velocizzare)
 - ▶ Aggiunge il FCB alla tabella dei file aperti di sistema
 - In ogni caso
 - ▶ Aggiunge alla tabella di file aperti del processo chiamante un nuovo elemento che punta alla entry nella tabella di file aperti di sistema
 - ▶ Incrementa il contatore delle aperture del file
 - ▶ La tabella dei file aperti contiene altri dati come la posizione di lettura/scrittura e la modalità di apertura (lettura, scrittura, esecuzione)
 - ▶ La open restituisce un puntatore alla entry nella tab dei file aperti per processo (file descriptor in Unix/Linux, handle in Windows)
- Alla chiusura del file LFS:
 - Decrementa il contatore e rimuove la entry dalla tabella del processo
 - Se il contatore di aperture vale 0 il FCB viene rimosso dalla tabella del SO

Apertura File

- Più in dettaglio:
- Open() passa il nome del file al Logical File System
- Prima verifica sulla system-wide open-file table se già aperto
 - Se aperto, open-file table per-processo aggiornato con pointer su entry di system-wide table
 - Se non è aperto, si cerca il nome nella struttura della directory
 - ▶ Parte della struttura è in cache per velocizzare
 - Appena trovato viene copiata la FCB in system-wide open-file table (con contatore di file aperti)
 - Poi si inserisce una entry nella open-file table per-processo
 - ▶ Puntatore alla system-wide
 - ▶ Offset di lettura/scrittura
 - ▶ Modalità di accesso
 - Open restituisce un puntatore alla open-file table per-processo
 - ▶ Tutte le operazioni su questo puntatore

Chiusura File

- Quando un processo chiude un file (Close())
 - Deallocata la entry nel open-file table per-processo
 - Decrementato il contatore sulla system-wide open-file table
 - Quando tutti hanno chiuso il file
 - ▶ La memoria secondaria è scritta
 - ▶ Si dealloca la entry nella system-wide table dei file aperti

- Molti sistemi tengono l'informazione sui file aperti in memoria principale
 - i blocchi associati sono in memoria secondaria
 - ▶ BSD Unix usa la cache con 85% di hit rate